

Symphony: Execution-Aware Consensus and Proposer Disaggregation

Ritual

April 20, 2026

Abstract

Blockchain consensus protocols optimize performance (latency and throughput) under the implicit assumption that transaction execution is computationally inexpensive. This assumption, however, fails for workloads that are resource-intensive, heterogeneous, and randomized. For randomized computations under independent private randomness, replicated state machine execution fails as a correctness mechanism: honest replicas evaluating the same randomized function with independent random coins produce divergent outputs. Proposers in current blockchain instantiations have extreme discretionary (and unilateral) power over the set of transactions in a block and their ordering. We present Symphony, an execution-aware consensus framework that disaggregates this bundle of powers (multi-laterally) into protocol-enforced inclusion, exclusion, timing, and ordering guarantees, each enforced as a conjunct of the block validity function verified in the consensus critical path. Delegated execution proceeds via verifiable computation over a product lattice of non-interactive proof systems with heterogeneous soundness-completeness-latency profiles. Distributed proof generation exploits the algebraic structure of computational objects (compositional hierarchy, parameter symmetry, repeated sub-computation), while distributed verification assigns proof shards to validator committees via deterministic sampling. We characterize the class of consensus protocols supporting extended external validity as those providing predictable validity, derive committee safety bounds via hypergeometric analysis under uniform staking, prove soundness composition for independent proof systems, and formalize the tradeoff between transaction-level censorship resistance and liveness degradation in the presence of proposer disaggregation.

Contents

1	Introduction	4
1.1	The Proposer Powers	4
1.2	The Execution-Cost Inversion	4
1.3	Randomness as a Structural Barrier	4
1.4	Contributions	5
1.5	Paper Organization	6
2	Formal Framework	6
2.1	System Model	6
2.2	Randomized Computation and the Limits of Replication	6
2.2.1	Hardware Non-Reproducibility	7
2.2.2	Algorithmic Randomness	7
2.2.3	Combined Effect	7
2.3	Non-Interactive Proof Systems	9
2.4	Product Lattice over Proof Systems	10
2.5	Predictable Validity	11

2.6	Proposer Power Bundle	12
2.7	Extended State Machine	13
3	Superposition of Execution Models	14
3.1	Replicated Execution (Deterministic Path)	14
3.2	Delegated Execution (Randomized / Resource-Intensive Path)	15
3.3	Superposition over Shared State	16
3.4	Two-Phase Delegated Execution via Saga Semantics	17
4	Verifiable Computation with Distributed Proving and Verification	18
4.1	Verifiable Delegated Computation	19
4.2	Structure-Exploiting Workload Decomposition	19
4.2.1	Algebraic Structure of Computational Objects	19
4.2.2	Formal Decomposition	20
4.2.3	Reference Decompositions: Transformers	22
4.2.4	Verification Units	23
4.3	Distributed Verification via Committee Sampling	23
4.3.1	Committee Assignment	23
4.3.2	Specialization	23
4.3.3	Quorum and Quorum-Skip	23
4.3.4	Security-Efficiency Tradeoff	24
4.3.5	Prover-Friendly Design Point	24
5	Multi-System Verification Model	25
5.1	Lattice Structure and Verification State	25
5.2	Soundness-Completeness-Latency Pareto Frontier	25
5.3	Failure Correlation Analysis	26
5.4	Monotonic State Advancement	26
5.5	Temporal Consistency	27
5.6	Irrevocable Execution with Economic Safety	27
5.7	Application Safety Declarations	28
5.8	Proof System Degradation Protocol	28
6	Extended External Validity	29
6.1	Extended Block Validity	30
6.2	Power Guarantees as Validity Conjunct	30
6.3	Predictable Validity Requirement	31
6.4	Censorship Resistance / Liveness Tradeoff	31
6.4.1	Setup	31
6.4.2	Block Construction Feasibility	32
6.4.3	Liveness under Power Guarantees	32
6.5	Predicate Evaluation	33
6.5.1	Default: Full Deterministic Evaluation	33
7	Inclusion and Exclusion Guarantees	33
7.1	Inclusion Guarantees	33
7.1.1	Comparison to Existing Mechanisms	34
7.2	Exclusion Guarantees	34
7.2.1	Use Cases	35
7.2.2	Comparison to Existing Mechanisms	36
7.3	Interaction between inclusion and exclusion	36
7.3.1	Conflict Detection	36

7.3.2	Runtime Conflict Resolution	37
7.4	Integration with the Verification Lattice	37
8	Application-Defined Ordering Guarantees	38
8.1	Transaction Ordering as Application-Layer Authority	38
8.2	Formal Definition	39
8.3	MEV Surface Analysis	40
8.4	Block Building Challenges and Mitigations	41
8.4.1	Finalized-State and Non-Finalized-State Ordering	42
9	Safety and Liveness	43
9.1	Safety	44
9.1.1	Replicated Execution	44
9.1.2	Delegated Execution: Single Proof System	44
9.1.3	Delegated Execution: Multiple Independent Proof Systems	45
9.1.4	Delegated Execution: Correlated Proof Systems	45
9.1.5	Distributed Verification: Hypergeometric Committee Bounds	45
9.2	Liveness	47
9.2.1	SMR Liveness	47
9.2.2	Executor Liveness	47
9.2.3	Prover Liveness	48
9.2.4	Liveness Under Inclusion & Exclusion Guarantees	48
9.2.5	Liveness Under Ordering Guarantees	49
10	Discussion	49
10.1	Limitations	49
11	Conclusion	50

1 Introduction

Blockchain state machine replication (SMR) protocols have converged on optimizing performance, namely latency and throughput. Recent work has driven finality times down and block production up: Solana’s Alpenglow [60], MonadBFT [63], Simplex [61], and Minimmit [62] reduce latency; DAG-based protocols (Narwhal [29], Bullshark [75]) decouple dissemination from ordering to achieve higher throughput. The implicit assumption underlying this trajectory is that transaction execution is cheap relative to SMR.

This assumption is breaking as increasingly off-chain compute comes on-chain. Workloads that are resource-intensive (requiring GPUs, hundreds of gigabytes of memory, minutes to hours of compute), heterogeneous (different transactions requiring fundamentally different hardware), and randomized (producing different outputs on each execution) do not fit the replicated execution model on which blockchain SMR was built.

1.1 The Proposer Powers

Block proposers in existing protocols exercise several distinct powers that are often treated as atomic: which transactions to include, which to exclude, how to order them, and when to propose. Recent Ethereum discussions have explored unbundling proposer or validator roles at a high level [84, 85]. However, most concrete protocol proposals still intervene on individual powers in isolation and for particular objectives, rather than offering a general framework in which these powers are explicitly separated and can be independently constrained or reassigned. For example, Proposer-Builder Separation [23] shifts block construction to a builder, but still transfers the relevant bundle largely as a unit. Protocol-Enforced Proposer Commitments (PEPC) push further in this direction by allowing proposers to make more general commitments about block contents that are enforced by the protocol, rather than merely outsourcing block construction wholesale [86]. Inclusion lists such as EIP-7547 [35] constrain omission discretion, order-fairness protocols [57] constrain sequencing through a global ordering rule, while encrypted mempools [74] reduce the proposer’s informational advantage without redistributing decision authority. Symphony generalizes this design space by transferring the decisions about block contents to on-chain applications and the users interacting with them at the protocol level.

1.2 The Execution-Cost Inversion

When SMR was the bottleneck, execution was an afterthought: all replicas could afford to execute all transactions because transactions were lightweight (token transfers, storage updates, simple contract calls), and the cost of reaching agreement dominated. For the workload class that motivates Symphony, this relationship inverts. Neural network inference with billions of parameters requires tens to hundreds of gigabytes of GPU memory and seconds to minutes of compute per forward pass; Monte Carlo simulations, probabilistic optimization, and other resource-intensive computations have similar profiles. Execution latency exceeds SMR latency by three to six orders of magnitude. Replicated execution of these workloads is not merely expensive: it requires every validator to possess specialized hardware, undermining the decentralization properties that replicated execution was designed to preserve.

1.3 Randomness as a Structural Barrier

The cost argument alone would motivate optimization but not architectural change. The structural barrier is randomness, which enters through two independent channels.

Hardware non-reproducibility. Replicated state machine execution [73] assumes that given the same input and pre-state, all correct replicas compute the same post-state. For GPU-accelerated workloads, however, this assumption fails at the hardware level. Floating-point accumulation in parallel GPU threads is non-associative: the order in which partial sums are reduced depends on thread scheduling, which varies across architectures and even across runs on the same device [54]. Two honest replicas executing the same neural network forward pass with the same weights, the same input, and the same random seed on different GPU hardware will produce bit-different outputs. This is not a software bug or a randomness problem; it is a consequence of IEEE 754 arithmetic under parallel reduction. Deterministic GPU modes exist but impose severe performance penalties (10–30× slowdown) and are not universally available, defeating the purpose of hardware acceleration. No amount of seed-sharing or protocol-level coordination eliminates hardware-level divergence across heterogeneous validators.

Algorithmic randomness. Independently of hardware, many target workloads are intentionally randomized. Neural network inference with temperature sampling ($T > 0$), Monte Carlo simulation, and probabilistic optimization all map inputs to *distributions* over outputs. When replicas sample independently, they produce different outputs not because any replica is faulty but because the function is inherently randomized. A standard workaround—deriving shared randomness from a beacon or VRF seed—makes the computation deterministic given the seed, but re-exposes it to hardware non-reproducibility and makes the output predictable from the public seed.

Consequence. Either source of randomness, alone, breaks replication. Together they make it untenable for the target workload class. Correctness requires proving that a given output is consistent with the computation, not comparing outputs across replicas. This is the domain of verifiable computation [42,81]. Section 2.2 formalizes the distinction and defines the randomness models Symphony supports.

1.4 Contributions

Symphony is an execution-aware SMR framework organized around a single principle: *protocol-enforced guarantees*. This paper makes the following contributions:

1. **Protocol-enforced guarantees.** The set of blocks that a proposer can build is constrained in order to provide guarantees on *inclusion*, *exclusion*, and *sequencing* to applications and users. Each guarantee is enforced as a conjunct of the block validity function via extended external validity [24].
2. **Superposition of execution models.** A single state machine simultaneously supports replicated execution (for deterministic, lightweight workloads) and delegated execution (for randomized or resource-intensive workloads) over shared state. Delegated execution follows a saga pattern [40] with no-rollback semantics and economic safety currently, but will in future work.
3. **Product lattice verification.** Delegated computations are verified through a product lattice [15] of non-interactive proof systems [17] with heterogeneous soundness-completeness-latency profiles. Applications declare safety requirements as upsets in the lattice.
4. **Distributed proving and verification.** Proof generation exploits the algebraic structure of computational objects (compositional hierarchy, parameter symmetry, repeated sub-computation) for workload-level decomposition. Verification is distributed across validator committees via deterministic sampling [43].

5. **Formal results.** We prove that predictable validity (the proposer can compute the post-confirmation state) is necessary for extended external validity with current-state predicates, derive committee safety bounds via hypergeometric analysis, prove soundness composition for independent proof systems under explicit independence assumptions, and formalize the censorship-resistance/liveness tradeoff.

Each contribution builds on established techniques (external validity [24], verifiable computation [42], saga transactions [40], hypergeometric sampling [43]). The novelty is in their composition and the unified enforcement model, not in any individual primitive.

1.5 Paper Organization

Section 2 establishes the formal framework: system model, randomized computations, proof systems, product lattice, predictable validity requirement, and block-content guarantees. Section 3 describes the superposition of replicated and delegated execution. Section 4 presents verifiable computation with structure-exploiting distributed proving and committee-based verification. Section 5 develops the product lattice verification model. Section 6 defines extended external validity and derives the CR/liveness tradeoff. Section 7 instantiates inclusion and exclusion guarantees. Section 8 instantiates ordering guarantees. Section 9 establishes safety and liveness. Section 10 discusses limitations and future work. Section 11 concludes.

2 Formal Framework

This section establishes the formal foundations on which Symphony is built. We define the system model, characterize the class of computations that require delegated execution, introduce the proof systems and product lattice used for verification, identify the structural property of SMR protocols that enables extended external validity, and present the protocol-enforced guarantees over inclusion, exclusion, and intra-block ordering of transactions.

2.1 System Model

We consider a distributed system of n stake-weighted replicas (validators), of which at most $f < n/3$ are Byzantine. The system operates under partial synchrony [34]: there exists a Global Stabilization Time (GST) and a message delay bound Δ , both unknown to the protocol, such that after GST all messages between correct replicas are delivered within Δ . Before GST, messages may be delayed arbitrarily but are not lost.

Each replica i holds stake $w_i > 0$ with $\sum_{i=1}^n w_i = W$. A set of replicas S constitutes a *quorum* if $\sum_{i \in S} w_i > 2W/3$. Byzantine replicas control at most $f_W < W/3$ total stake.

We analyze safety under both uniform staking ($w_i = W/n$ for all i) and non-uniform staking distributions. Committee safety bounds (Section 9.1.5) are derived under the uniform assumption and then relaxed.

The system runs a *state machine replication* (SMR) protocol [73]. SMR (equivalently, atomic broadcast) is the task of maintaining a totally-ordered log of state transitions such that all correct replicas agree on the same sequence of transitions and apply them in the same order. Formally: given a deterministic state transition function $\delta : \mathcal{S} \times \mathcal{T} \rightarrow \mathcal{S}$, SMR guarantees that (1) all correct replicas deliver the same sequence of transactions, and (2) the resulting state is uniquely determined by applying δ to the initial state in that order. We extend the standard SMR model with additional state components defined in Section 2.7.

2.2 Randomized Computation and the Limits of Replication

A *workload* is a triple $W = (\Theta, F, D)$ where:

- $\Theta \subseteq \mathbb{R}^d$ is a parameter space (finite-dimensional, possibly very large),
- $\mathcal{X}$ is the input space and $\mathcal{Y}$ is the output space,
- $F : \Theta \times \mathcal{X} \rightarrow \mathcal{D}(\mathcal{Y})$ maps parameters and input to a probability distribution over outputs,
- $D = (c_{\text{compute}}, c_{\text{memory}}, c_{\text{time}})$ is a resource descriptor.

Replicated execution assumes deterministic state transitions: given the same input and pre-state, all correct replicas compute the same post-state [73]. Two independent sources of randomness violate this assumption for the target workload class.

2.2.1 Hardware Non-Reproducibility

In practice, re-execution often does not yield identical results across machines. Floating-point accumulation in parallel GPU threads is non-associative: the order of reduction depends on thread scheduling and hardware-specific reduction trees [54]. Two correct replicas executing the same neural network forward pass with identical weights, input, and random seed on different GPU architectures can produce bit-different outputs—a consequence of IEEE 754 arithmetic, not a software defect. Neural network inference on GPUs exhibits this for any model with parallel reductions (attention, layer normalization). Deterministic GPU modes exist but impose severe performance penalties and are not universally available. No protocol-level coordination restores bit-exact agreement across heterogeneous validators; replication fails because the hardware cannot produce identical outputs. This necessitates delegated execution with external verification.

2.2.2 Algorithmic Randomness

Independently of hardware, many target workloads are intentionally randomized: they sample from distributions rather than computing deterministic functions. Neural network inference with temperature sampling, Monte Carlo simulation, and randomized optimization all map inputs to distributions over outputs. When replicas sample independently, they produce different outputs—not because any replica is faulty, but because the function is inherently randomized. Any comparison scheme that requires identical outputs across replicas will fail. This motivates delegated execution: correctness requires proving that a given output is consistent with the computation, not comparing outputs across replicas.

Definition 2.1 (Deterministic Workload). *A workload W is deterministic if for all $\theta \in \Theta$ and $x \in \mathcal{X}$, the distribution $F_\theta(x)$ is a point mass: there exists a unique $y \in \mathcal{Y}$ such that $\Pr[F_\theta(x) = y] = 1$.*

Definition 2.2 (Algorithmically Randomized Workload). *A workload W is algorithmically randomized if there exist $\theta \in \Theta$ and $x \in \mathcal{X}$ such that $F_\theta(x)$ has positive Shannon entropy: $H(F_\theta(x)) > 0$.*

A shared seed derived from a beacon or VRF [43] makes the computation deterministic given the seed, addressing algorithmic randomness. But this re-exposes the workload to hardware non-reproducibility (Section 2.2.1) and makes the output predictable from the public seed.

2.2.3 Combined Effect

Either source of randomness alone breaks replication. Hardware non-reproducibility is the more fundamental barrier: it is intrinsic to the execution substrate and cannot be eliminated by protocol design. Algorithmic randomness is addressable by shared seeds but at the cost

of predictability and with the residual problem that the “deterministic” replay is still not bit-reproducible across hardware.

For workloads exhibiting either or both forms of randomness, correctness requires proving that a given output y is consistent with $F_\theta(x)$ rather than comparing outputs across replicas. This is the domain of verifiable computation [42, 81].

Randomness model. Symphony’s delegated execution path supports three randomness models, declared when registering a delegated computation (Section 3.4):

1. *Sealed-seed execution (primary).* The user (or the protocol) selects a random seed r and encrypts it to the executor’s TEE attestation-derived public key: $e = \text{Enc}(\text{pk}_{\text{TEE}}, r)$. The ciphertext e is committed on-chain when the delegated computation is registered. The TEE decrypts r inside the enclave, evaluates $y = F_\theta(x; r)$, and produces an attestation covering the tuple (M, θ, x, e, y) , where M is the workload identity (the TD measurement in Intel TDX, i.e., the hash of the trust domain’s initial state). The executor—the machine operator outside the enclave—never observes r in plaintext. Grinding is prevented: the executor cannot enumerate seeds because the seed is encrypted to the TEE, and cannot selectively suppress outputs because the attestation is produced atomically with the computation. The workload identity M ensures the enclave binary is immutable post-attestation, so the executor cannot modify the program to leak r or branch on the output before reporting it.

This model provides *distributional integrity under the TEE soundness assumption*: the output y is the result of faithfully executing $F_\theta(x; r)$ for a seed r that the executor could not observe, choose, or grind over. The soundness parameter s_1 of the hardware attestation (Section 2.3) bounds the probability that this guarantee is violated. If the TEE is compromised (side-channel attack, firmware vulnerability), the attacker can decrypt r and grind; this failure mode is already captured by s_1 .

2. *Executor-chosen randomness.* The executor selects r freely and proves $y = F_\theta(x; r)$ for the chosen r . The proof establishes *support membership*: y is in the range of $F_\theta(x)$. It does not establish distributional fairness—the executor can grind over r to bias the output. This model is appropriate when the application cares only that y is a feasible output, not that it was sampled fairly (e.g., selecting any valid completion of a prompt).
3. *Protocol-derived randomness.* The seed r is derived from a protocol-provided source (beacon, VRF) fixed before execution. The executor proves $y = F_\theta(x; r)$ for the specific r . This model provides deterministic replay and eliminates executor grinding, but the output is predictable given r and x .

Under model (1), the “valid sample” claim is backed by the TEE’s execution integrity: the seed was user-chosen and executor-blind, and the program was measured and immutable. Under model (2), the claim is limited to support membership. Under model (3), it is deterministic correctness.

User-chosen vs. protocol-chosen seeds. In model (1), the seed originator matters. If the user provides r , they can choose it adversarially to manipulate their own output; this is acceptable when the output affects only the user’s own state (e.g., a user requesting inference on their own prompt). When the output affects shared state (e.g., oracle prices, risk assessments consumed by other contracts), the seed should originate from a source the user does not control: a randomness beacon, a VRF, or a commit-reveal scheme where the seed is committed before the user’s input is known. The choice of seed originator is declared at registration time alongside the randomness model.

2.3 Non-Interactive Proof Systems

Symphony’s verification layer is built on proof systems. We adopt the standard definition [17, 46] with a deliberate architectural restriction to non-interactive systems [17].

Definition 2.3 (Non-Interactive Proof System). *A non-interactive proof system for a relation $\mathcal{R} \subseteq \mathcal{X} \times \mathcal{W}$ is a pair of probabilistic polynomial-time algorithms $(\text{Prove}, \text{Verify})$ such that:*

- $\text{Prove}(x, w) \rightarrow \pi$: *given statement x and witness w , produces a proof π in a single message.*
- $\text{Verify}(x, \pi) \rightarrow \{0, 1\}$: *accepts or rejects without further interaction.*

Definition 2.4 (Completeness and Soundness). *A non-interactive proof system $V = (\text{Prove}, \text{Verify})$ satisfies:*

- *Completeness with parameter $c_V \in [0, 1]$: for all $(x, w) \in \mathcal{R}$,*

$$\Pr[\text{Verify}(x, \text{Prove}(x, w)) = 1] \geq c_V.$$

- *Soundness with parameter $s_V \in [0, 1]$: for all x such that no w satisfies $(x, w) \in \mathcal{R}$, and for all (possibly adversarial) proofs π^* ,*

$$\Pr[\text{Verify}(x, \pi^*) = 1] \leq s_V.$$

We extend the standard definition with one parameter relevant to the blockchain setting:

Definition 2.5 (Extended Proof System). *An extended proof system is a tuple*

$$V = (\text{Prove}, \text{Verify}, s_V, c_V, \ell_V)$$

where s_V and c_V are soundness and completeness parameters, and ℓ_V is the expected proving latency (time for Prove to produce π).

Latency is an engineering parameter. The purpose of this extension is to make the soundness-completeness-latency tradeoff explicit. Failure correlation between proof systems (whether compromise of one implies compromise of another) is a property of proof system *pairs*, not of individual systems, and is analyzed in Section 5.3.

The non-interactivity constraint. Symphony restricts to non-interactive proof systems. This excludes interactive verification schemes, including optimistic fraud proofs as used in Optimism [66] and Arbitrum [3]. The restriction is a deliberate architectural choice with three consequences:

1. *No synchrony assumptions in verification.* Interactive fraud proofs require a synchrony assumption: the honest challenger must submit a fraud proof within a bounded challenge window. This introduces a synchrony requirement beyond what the underlying SMR protocol demands (which operates under partial synchrony [34]). Non-interactive proof systems require no additional synchrony: a proof is valid or invalid regardless of when it is checked.
2. *Simpler peer-to-peer layer.* Non-interactive proofs are data: they can be gossiped, stored, and verified independently by any replica at any time. Interactive verification requires a protocol layered on top of the SMR protocol, with its own liveness and safety requirements. Symphony avoids this additional protocol complexity.

3. *Bounded verification latency.* The time from proof generation to verification acceptance is $\ell_V + \delta$, where δ is the network propagation delay. There are no interactive rounds, no challenge periods, and no dispute games. For fraud proofs, the equivalent latency includes the full challenge period (typically days in production systems).

The tradeoff is that some computations efficiently verifiable via interactive protocols may not be efficiently verifiable non-interactively [45]. In practice, the proof systems relevant to Symphony—hardware attestations, SNARKs [48], STARKs [12]—are all non-interactive.

2.4 Product Lattice over Proof Systems

Standard SMR treats transactions as opaque: validity is structural (signatures, well-formedness). Symphony’s delegated execution requires application-specific verification (ZK proofs, hardware attestations, or combinations thereof), and different applications have different safety needs. The product lattice allows users to dictate their own safety requirements: an application can require “at least one hardware attestation and one ZK proof,” or “any two of three proof systems,” without the protocol imposing a single global choice. This flexibility is the motivation for the construction below.

Given m non-interactive proof systems $\{V_1, \dots, V_m\}$, we define the *verification state* of a delegated computation result o as a vector:

$$\sigma(o) = (v_1, \dots, v_m) \in \{0, 1\}^m$$

where $v_i = 1$ if and only if proof system V_i has verified the result (i.e., a quorum of assigned validators has accepted the proof under V_i).

The set $\{0, 1\}^m$ with componentwise ordering forms a boolean hypercube with a natural partial order. For two states $\sigma = (v_1, \dots, v_m)$ and $\sigma' = (v'_1, \dots, v'_m)$:

$$\sigma \leq \sigma' \iff v_i \leq v'_i \text{ for all } i \in [m].$$

The lattice has 2^m elements, a bottom element $\perp = (0, \dots, 0)$ (no proof system has verified), and a top element $\top = (1, \dots, 1)$ (all proof systems have verified). The meet and join operations are componentwise min and max, respectively.

Boolean hypercubes with componentwise ordering are a standard construction in computer science, appearing in information flow security, abstract interpretation, and distributed knowledge models.

Definition 2.6 (Application Safety Requirement). *An application’s safety requirement is an upset (upward-closed set) $U \subseteq \{0, 1\}^m$: if $\sigma \in U$ and $\sigma \leq \sigma'$, then $\sigma' \in U$. A result o is accessible to an application with requirement U if and only if $\sigma(o) \in U$.*

Upsets are the natural formalism for “at least this much verification.” For example, with $m = 3$ proof systems (hardware attestation, SNARK, STARK):

- “At least one hardware attestation and one cryptographic proof” defines the upset $U = \{(1, 1, 0), (1, 0, 1), (1, 1, 1)\}$ and its upward closure.
- “At least two of three” defines a different upset.
- “All three” defines the singleton upset $\{(1, 1, 1)\}$.

Proposition 2.7 (Soundness Composition for Independent Proof Systems). *Let $V_1, \dots, V_m$ be proof systems with independent failure modes and soundness parameters $s_1, \dots, s_m$. The probability that an invalid result is accepted by all m systems is at most $\prod_{i=1}^m s_i$.*

Proof. By independence, the event that V_i accepts an invalid result is independent across i . The joint acceptance probability is the product of marginal acceptance probabilities, each bounded by s_i . $\square$

For correlated proof systems (e.g., multiple hardware attestations from the same vendor), independence does not hold and the joint analysis requires explicit modeling of the correlation structure, as standard in reliability theory [10]. The lattice structure makes the independence assumptions explicit: applications that require high combined soundness should specify proof systems with independent failure modes, which correspond to orthogonal dimensions in the lattice.

2.5 Predictable Validity

Extended external validity (Section 6) requires that the block proposer and all correct validators agree on the state against which validity predicates are evaluated. When a proposer’s turn comes to propose a block, they can exactly compute the state resulting from confirmation of their block—that is *predictable validity*. We formalize this as a property of the underlying SMR protocol.

Definition 2.8 (Predictable Validity). *An SMR protocol provides predictable validity if the following holds: when a correct proposer P constructs a block B at time t , there exists a unique state S (the state P is building on) such that:*

1. *S is derived from the committed prefix of the ledger from P ’s view. (Other validators may lag behind P ’s view of the committed prefix; the key is that P builds on a well-defined prefix.)*
2. *P can compute S from its local information at time t . For every correct validator V that receives B , V can verify that S is the correct pre-state (derived from the committed prefix) and that B is valid when applied to S —without needing to re-execute delegated computations in B , which are verified via proofs.*
3. *For any predicate $p : \mathcal{S} \rightarrow \{0, 1\}$ over the state space, $p(S)$ is well-defined and yields the same result for P and all correct validators.*

This property is a direct consequence of total-order state machine replication [73]: when a proposer is about to propose a block, they can exactly compute the state resulting from confirmation of their block—they know the contents they are building on. This is commonly called *predictable validity* in the SMR literature.

Proposition 2.9 (Predictable Validity Requirement for Extended External Validity). *Let Π be an SMR protocol and let $p : \mathcal{S} \rightarrow \{0, 1\}$ be a state predicate used in the block validity function. If Π does not provide predictable validity, then there exist executions in which a correct proposer P constructs a block B that P considers valid (because $p(S_P) = 1$ under P ’s view of the state) but a correct validator V considers invalid (because $p(S_V) = 0$ under V ’s view), or vice versa.*

Proof. Suppose Π does not provide predictable validity. Then there exist a correct proposer P at time t and a correct validator V at time $t' > t$ such that the states S_P and S_V derived from their respective local information differ: $S_P \neq S_V$. Since p is an arbitrary predicate, there exists a p that distinguishes S_P and S_V (e.g., $p(S) = \mathbf{1}[S = S_P]$). For this p , $p(S_P) = 1$ but $p(S_V) = 0$. If p is a conjunct of the block validity function, the proposer constructs a block that it believes is valid (it includes transactions mandated by p), but the validator rejects the block (because p is not satisfied under S_V). This contradicts the agreement property of the SMR protocol: correct replicas disagree on the validity of a block proposed by a correct proposer. $\square$

Classification of SMR protocols. We classify existing SMR protocols by this property:

Protocol family	Predictably valid?	Staleness
Single-leader, linear (CometBFT [21], Hot-Stuff [83])	Yes	0
Single-leader, pipelined (MonadBFT, Simplex [61])	Yes	0
Leaderless, metastable (Avalanche [71])	Probabilistic	Convergence-dep.
DAG-based (Bullshark [75], Narwhal [29])	Over committed prefix	$\leq$ commit interval

The key parameter is *staleness* [9]: the gap between the latest state and the predictably valid state available at proposal time. Linear protocols have staleness 0: the proposer’s predecessor state is precisely the latest committed state. DAG-based protocols have staleness bounded by the commit interval. The state corresponding to the last committed anchor round is predictably valid, but more recent state depends on future vertices and is not yet determined.

Predicates over the current state (the common case for applications: “trigger when price crosses threshold,” “include when oracle updates”) require staleness 0. Predicates that tolerate bounded staleness δ (“trigger when price crossed threshold at some point in the last δ rounds”) can operate on DAG-committed prefixes. This is not a binary classification but a parameterized one.

2.6 Proposer Power Bundle

In a leader-based SMR protocol, a proposer has discretion over which valid block to submit for a given pre-state. Most protocols impose basic constraints on block validity, e.g. that the block is well-formed, all transaction signatures are valid, and the parent hash is correct. Outside of these basic checks, the proposer has full power to determine the contents of the block. Some existing protocols further constrain the block proposer’s discretion over the block. For example, in Ethereum, all included transactions must pay at least a base fee that is burned by the protocol (EIP-1559) for a block to be considered valid. In this way, the protocol limits the ability of the proposer to include arbitrary transactions, even if the block has space remaining.

Some other existing modifications further limit the proposer’s abilities (both in- and out-of-protocol):

Mechanism	Action
Inclusion lists [35]	Enforce transaction inclusion in the next block
Hyperliquid [7]	Enforces cancel prioritization over take orders
MEV bundles [39]	Enforce atomicity of transaction bundles

Note that these constraints differ from the baseline validity checks on blocks in that they are specifically designed to provide additional features to applications and users. In other words, enforced restrictions of the proposer’s action space can be interpreted as guarantees that applications and users find economically valuable. Symphony supports a much broader class of such guarantees by streamlining their enforcement through consensus and allowing applications and users to register according to their needs.

In this paper, we instantiate three categories of guarantees (Table 1). These are grouped by the block-construction dimension they primarily restrict and are use-case driven rather than fundamental, as the families can overlap and may conflict (see e.g. Section 7.3).

Inclusion guarantees promise that a designated transaction will be present whenever a state predicate is true. An inclusion constraint consists of a transaction T and a predicate $P : \mathcal{S} \rightarrow \{0, 1\}$, and if $P(S) = 1$, then any block B such that $T \notin B$ is invalid. Observe that

Guarantee	Typical registrant	Enforcement
Inclusion	User	Block invalid if triggered tx absent
Exclusion	Protocol / User / App	Block invalid if excluded tx present
Ordering	Application	Block invalid if order violates $(\pi_C, \{\tau_i\})$

Table 1: Families of guarantees provided by Symphony.

inclusion guarantees are attached to a specific transaction, and are determined entirely as a function of the pre-state, with no other dependence on the content of block B . See Section 7.1 for a more in-depth treatment of inclusion guarantees.

Exclusion guarantees promise that certain transactions will not be present. They consist of a predicate $Q : \mathcal{S} \times \mathcal{T} \rightarrow \{0, 1\}$, where if $Q(S, T) = 1$ for state S and transaction T , then any block B containing T is invalid. Exclusion guarantees can be registered by different parties: Users can attach exclusion guarantees to a specific transaction to prohibit its inclusion as a function of state (e.g., a user can submit a transaction to trade with an AMM, but require that it cannot be included if the AMM price is outside a certain desirable range). Applications can apply exclusion guarantees to all transactions that interact with the application (e.g., a trading application can implement a circuit-breaker when oracle staleness exceeds a threshold). Finally, the protocol itself can use exclusion guarantees (e.g., quarantine during proof system degradation). See Section 7.2 for a more in-depth treatment of exclusion guarantees.

Ordering guarantees make promises about the relative ordering of the subsequence of transactions targeting an application in a block. A contract C declares an ordering policy $(\pi_C, \{\tau_i\})$: a finite ordered list of recognized call sequences over C ’s functions, with optional tiebreakers τ_i evaluated on finalized state for intra-priority ordering. Transactions whose call sequences match a recognized pattern must appear in the block in the order specified by π_C ; transactions with unrecognized call sequences are automatically deprioritized and placed below all recognized transactions. See Section 8 for a more in-depth treatment of ordering guarantees.

Each guarantee has distinct implications for the incentives of participants and the consensus mechanism. Inclusion guarantees strengthen censorship resistance at the cost of liveness (more mandatory inclusions increase the proposer’s critical-path work). Exclusion guarantees enable application-level safety controls but introduce a potential griefing vector (malicious exclusion predicates). Ordering rights constrain the proposer’s intra-application ordering for adopting contracts but do not constrain inter-application ordering (where the application’s transaction subsequence sits within the block), inclusion decisions (which transactions targeting the contract enter the block), or interleaving with other contracts’ transactions. MEV shifts from the intra-application ordering surface to the inclusion, interleaving, and composability surfaces. Each requires its own incentive analysis (akin to [72]).

2.7 Extended State Machine

The standard SMR model treats transactions as opaque: the validity condition checks structural properties (signatures, well-formedness) and execution correctness (replicated transactions are re-executed by all validators). Symphony introduces flexibility into the state and validity layers: the state includes pending delegated computations, active triggers, and ordering policies; the validity function references these via extended external validity. This subsection defines the extended state.

Symphony extends the standard SMR model [73] with additional state components:

Definition 2.10 (Symphony State). *The state S consists of:*

- Execution state S_{exec} : *account balances, contract storage (standard).*
- Async result registry S_{async} : *for each pending or settled delegated computation, a record $(id, \theta, x, status, y, \sigma)$ where $status \in \{PENDING, SETTLED, DISPUTED, EXPIRED\}$, y is the output (if settled), and $\sigma \in \{0, 1\}^m$ is the verification state.*
- Trigger pools: *a set of active inclusion triggers $\mathcal{I} = \{(P_i, T_i, TTL_i)\}$ and exclusion triggers $\mathcal{E} = \{(Q_j, TTL_j)\}$, where each Q_j is scoped to transactions targeting the registering contract.*
- Ordering policies: *a map from contract addresses to ordering policies $\{(\pi_C, \{\tau_i\})\}_{C \in \mathcal{C}}$.*
- Verification state: *per-proof-system attestations and committee assignments.*

State transitions fall into the following categories:

1. *Replicated execution*: deterministic state transitions executed by all correct replicas (standard EVM semantics).
2. *Async commit*: registers a delegated computation (id, θ, x) with status PENDING.
3. *Async settlement*: transitions a pending computation to SETTLED with output y and initial verification state σ .
4. *Verification attestation*: advances a component v_i of $\sigma(o)$ from 0 to 1 upon quorum attestation.
5. *Safety promotion*: transitions verification state when all required proof systems have verified (triggered via inclusion guarantees).
6. *Dispute*: transitions to DISPUTED when a proof system rejects a previously-attested result.
7. *Trigger registration/deregistration*: adds or removes inclusion/exclusion triggers and ordering policies.

The block validity function over this extended state is defined in Section 6.

3 Superposition of Execution Models

A blockchain state machine typically assumes a single execution model: all replicas execute all transactions deterministically. Symphony replaces this assumption with a *superposition* of two execution models operating over shared state within a single block. This is not lane separation (isolated execution environments with separate state) but genuine superposition: both models read and write the same state, and their interactions are mediated by the verification lattice.

This section describes the two execution modes, how they compose, and the saga-based mechanism for long-lived delegated execution.

3.1 Replicated Execution (Deterministic Path)

Deterministic state transition functions that fit within the block’s computational budget are executed by all correct replicas in the standard SMR manner [73]. This is identical to how Ethereum, Solana, and other replicated-execution blockchains process transactions today.

Formally, a replicated transaction tx applies a deterministic function $g : \mathcal{S} \rightarrow \mathcal{S}$ to the current state:

$$S' = g(S, tx)$$

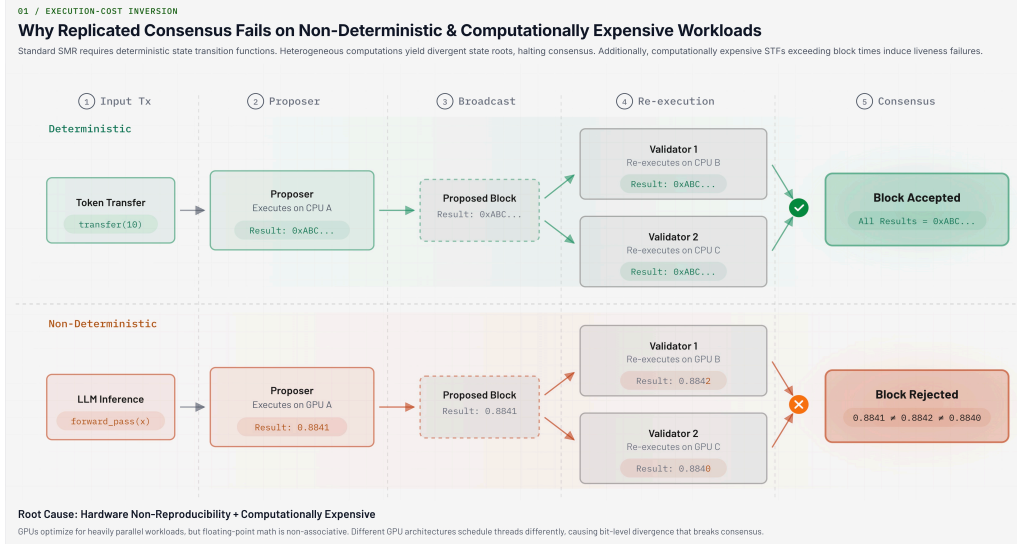


Figure 1: Execution-cost inversion. Standard SMR assumes deterministic, lightweight state transitions (top). Heterogeneous GPU workloads produce divergent state roots across replicas, breaking the replication assumption (bottom).

Every correct replica computes g independently and arrives at the same S' . Safety follows from the determinism of g and the agreement property of the SMR protocol (Theorem 9.1).

Replicated execution handles the vast majority of blockchain transactions: token transfers, AMM swaps, governance votes, contract deployments. These are deterministic, lightweight, and well-served by the standard model. Symphony does not modify or replace this path.

3.2 Delegated Execution (Randomized / Resource-Intensive Path)

Workloads that are randomized (Definition 2.2) or that exceed the per-block computational budget are executed by a single staked executor. The remaining replicas verify proofs of the execution rather than re-executing.

A delegated transaction for workload $W = (\Theta, F, D)$ with input x proceeds as follows:

1. **Executor selection.** A staked executor E with sufficient resources (matching the resource descriptor D) is selected. Selection may be deterministic (round-robin among qualified executors) or market-based, via a two-sided matching mechanism that accounts for heterogeneous user valuations and heterogeneous node costs [6].
2. **Execution.** E evaluates $y \sim F_\theta(x)$ inside the TEE, producing an output y and an initial proof (the hardware attestation π_1). Proofs for slower proof systems ($\pi_2, \dots, \pi_m$) are generated asynchronously and may arrive much later.
3. **Settlement.** E submits a settlement transaction containing (y, π_1) . The result enters the verification lattice at position $(1, 0, \dots, 0)$. Subsequent proofs π_i are submitted in separate transactions as they become available, each advancing the corresponding lattice dimension.
4. **Verification.** Each proof π_i is verified independently by its assigned committee (Section 9.1.5). Upon committee quorum for V_i , the lattice dimension v_i advances from 0 to 1. Proofs arrive and are verified on their own timescales: hardware attestation within milliseconds of execution, ZKPs minutes to hours later.

Other replicas never execute $F_\theta(x)$. They verify proofs, which is computationally cheaper, both for TEE and ZK proofs.

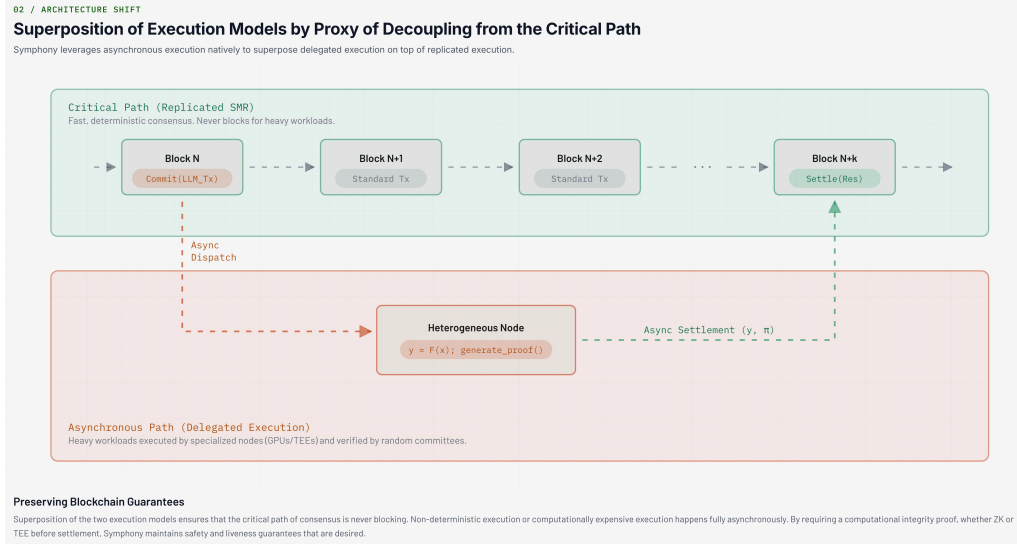


Figure 2: Superposition architecture. The critical path handles lightweight deterministic transactions via replicated execution. Resource-intensive or randomized workloads are delegated to a single executor and verified asynchronously by committees, merging back into shared state upon settlement.

The Fabric Execute-Order-Validate (EOV) architecture [2] is the closest architectural ancestor. Fabric’s endorsement policies play the role of proof systems, but endorsement policies are interactive (endorsing peers must be online and responsive) and based on identity rather than cryptographic soundness. Symphony replaces endorsement policies with non-interactive proof systems (Section 2.3), adds the product lattice for heterogeneous verification (Section 5), and enshrines intra-block guarantees via external validity (Section 6).

3.3 Superposition over Shared State

Both execution modes operate on the same state S within a single block. A block’s state transition is a composition:

$$\text{STF}_{\text{block}}(S) = \text{STF}_{\text{replicated}}(\text{STF}_{\text{delegated}}(S))$$

The ordering shown (delegated settlements processed before replicated transactions) is a simplification. In practice, the block applies transactions in their sequential order within the block, regardless of type. Each transaction’s type (replicated vs. delegated settlement/commit) determines which execution path processes it, but the ordering is the block’s transaction sequence, not a type-based partition. The composition is uniquely determined by the block contents and the transaction sequence, which is agreed upon by SMR.

The interactions between modes are explicit and mediated by the verification lattice:

Replicated reads delegated results. A replicated transaction can read the output of a settled delegated computation, subject to the constraint that the result’s lattice position meets or exceeds the application’s declared safety requirement (Definition 2.6). If the result has not yet reached the required lattice position, the read returns a sentinel value indicating “not yet verified at required level,” and the replicated transaction can branch on this.

Replicated triggers delegated execution. A replicated transaction can register an async commit for a delegated computation. For example, a smart contract receiving a price up-

date (replicated, deterministic) can trigger an ML-based risk assessment (delegated, resource-intensive). The trigger is mediated by the async result registry (Section 2.7).

Delegated settlement triggers replicated actions. When a delegated computation settles and its lattice position reaches a specified threshold, an inclusion trigger (Section 7.1) can fire a replicated transaction that consumes the result. This is the mechanism for “chaining” delegated and replicated computations.

Shared state, not shared execution. The two modes share state but not execution. Replicated transactions are executed by all correct replicas. Delegated transactions are executed by one executor and verified by committees. The state is the integration point; the execution paths are distinct. This is an unbundling of SMR from execution that is *within* a single protocol, not across protocols (as in the Celestia/rollup modular architecture [26]).

3.4 Two-Phase Delegated Execution via Saga Semantics

Delegated execution is inherently long-lived: proof generation for large workloads takes minutes to hours. The chain cannot block during this period. Symphony uses a two-phase pattern based on the saga model [40]:

Phase 1: Commit. An async commit transaction anchors the computation parameters on-chain:

- Computation identifier id.
- Parameters θ (or a hash commitment to parameters stored off-chain).
- Input x (or a hash commitment).
- Designated executor E (or an open assignment for market-based selection).
- Time-to-live TTL: maximum blocks until settlement.
- Proof system requirements: which dimensions of the lattice must be populated.

The result registry is updated: $(\text{id}, \theta, x) \mapsto \text{Pending}$. The executor’s bond for this computation is locked.

Phase 2: Settlement. In a future block (possibly many blocks later), the executor submits a settlement transaction:

- Output y .
- Proofs $\{\pi_1, \dots, \pi_m\}$ (some may be submitted later if proof generation is ongoing).
- Auxiliary data for verification (e.g., intermediate commitments for distributed verification).

The result status transitions to **Settled**, and verification begins. The lattice position starts at $\perp = (0, \dots, 0)$ and advances as proof systems verify.

Compensating action on failure. If the TTL expires without settlement:

1. The result status transitions to **Expired**. (This status is formally defined in the extended state machine, Definition 2.10, alongside **PENDING**, **SETTLED**, and **DISPUTED**.)
2. The executor’s bond is partially slashed ($\beta \cdot b$ for slashing parameter $\beta \in (0, 1]$).
3. The computation becomes eligible for rescheduling: any qualified executor can claim it.
4. Applications that had registered downstream triggers on this computation’s result are notified via a sentinel state, enabling compensating actions (e.g., falling back to a cached result, reverting to a safe default, or refunding the user).

No locks. Between commit and settlement, the chain advances freely. No state is locked. Other transactions can read, write, and modify any state except the result registry entry for the pending computation. Pending results are not readable as settled values; any transaction that attempts to read a pending result receives the sentinel value.

This differs from two-phase commit (2PC) [14] in a critical way: 2PC locks resources until the coordinator decides. Sagas release resources immediately and use compensating transactions on failure. For a blockchain where state is globally visible and blocks must be produced continuously, sagas are the appropriate model.

Stale-precondition risk (TOCTOU—Time-Of-Check to Time-Of-Use). Between the async commit at block N and the settlement at block $N + k$, other transactions may alter the state assumptions on which the delegated computation relies. The executor computed $y = F_\theta(x)$ against state S_N , but settlement occurs against state S_{N+k} , which may invalidate the relevance of y . Symphony does not enforce read-set consistency between commit and settlement at the protocol level; this is a deliberate design choice that shifts atomicity responsibility to the application layer. Applications requiring state consistency across the async gap should (a) declare the relevant state slots at commit time, (b) check for state drift in the settlement transaction, and (c) trigger compensating actions if drift exceeds tolerance. The default behavior—settling without read-set verification—is safe for computations whose correctness is independent of intermediate state changes (e.g., proving properties of the model weights, which do not change during the async gap), but is a “foot-gun” for computations that depend on volatile state (e.g., computing a liquidation amount based on prices at commit time).

Downstream chaining. Computations can be chained: the output of one delegated computation serves as input to the next, or triggers a replicated transaction. Chaining is mediated by inclusion triggers: “when result R reaches lattice position $\sigma_{\min}$, include transaction T .” This allows pipelines of mixed replicated and delegated execution without blocking the chain at any stage.

4 Verifiable Computation with Distributed Proving and Verification

Delegated execution requires that the network can verify a result without re-executing the computation. This section describes how Symphony implements verifiable computation [42, 81] with two key extensions: (a) distributed proof generation that exploits the algebraic structure of the computational objects, and (b) distributed verification via committee sampling.

4.1 Verifiable Delegated Computation

A single staked executor evaluates $F_\theta(x)$, producing output y and proofs $\{\pi_1, \dots, \pi_k\}$ for k registered non-interactive proof systems. This is standard verifiable computation [42] extended in two directions:

1. *Randomized computations.* For a randomized workload W (Definition 2.2), the proof establishes that y is consistent with $F_\theta(x)$ under the declared randomness model (Section 2.2). Under the primary sealed-seed model, the user encrypts a seed r to the TEE’s attestation-derived key; the enclave decrypts r , evaluates $F_\theta(x; r)$, and attests the result alongside the workload identity M and the on-chain ciphertext e . The executor never observes r and cannot grind. Under executor-chosen randomness, the proof shows $y = F_\theta(x; r)$ for some executor-selected r , establishing support membership but not distributional fairness. Under protocol-derived randomness, the proof shows $y = F_\theta(x; r)$ for a beacon/VRF seed r committed on-chain, establishing deterministic correctness. The ZKP (when present) proves the computation trace including r ; the TEE attestation covers the enclave environment, workload identity, and the ciphertext binding. Note that SNARK-friendly inference circuits operate over finite fields ($\mathbb{F}_p$), where arithmetic is exact and associative. Hardware non-reproducibility (Section 2.2.1) does not arise in the ZKP verification path; the only source of non-determinism within the circuit is the randomness input r .
2. *Multiple proof systems simultaneously.* The executor produces proofs for multiple proof systems, each with different soundness-completeness-latency profiles (Section 2.3). The proofs are independent: each proof system’s proof is generated and verified without reference to the others.

4.2 Structure-Exploiting Workload Decomposition

For large workloads, proof generation by a single machine is infeasible or impractically slow. Symphony distributes proof generation across multiple machines. The key insight is that the computational objects over which delegated STF’s operate are not arbitrary circuits — they exhibit algebraic structure that makes decomposition natural and efficient.

4.2.1 Algebraic Structure of Computational Objects

The workloads relevant to Symphony, including neural network inference, probabilistic simulation, and optimization, share structural properties that generic circuit-level decomposition [82] does not exploit:

Compositional hierarchy. A neural network is a composition of layers: $F = f_L \circ f_{L-1} \circ \dots \circ f_1$, where each $f_i : \mathbb{R}^{d_i} \rightarrow \mathbb{R}^{d_{i+1}}$ is a layer function (linear transformation followed by nonlinearity). Layers are natural partition boundaries: the interface between consecutive layers is an activation tensor of fixed, architecture-determined dimension (e.g., shape (s, d_{model}) for a transformer), not an arbitrary set of wire values.

Parameter symmetry. In transformer architectures [80], attention heads within a layer are structurally identical sub-computations operating on different parameter slices. If a layer has h attention heads, each computing $\text{Attention}(Q_i, K_i, V_i)$ for $i \in [h]$, the heads are interchangeable up to their parameters. This symmetry enables balanced partitioning: each head can be assigned to a different prover with guaranteed uniform workload.

Repeated sub-computation. Residual blocks in ResNets, recurrent cells in RNNs, and timesteps in diffusion models are iterated applications of the same or similar structure. Regularity enables uniform partition sizing and predictable proving costs.

These structures are well-studied in the parallel and distributed computing literature [32, 55]. Applying them to the distributed *proving* problem (rather than the distributed *execution* problem) yields partition boundaries aligned with the object’s algebraic structure.

4.2.2 Formal Decomposition

Graph partition model. Represent the computation as a directed acyclic graph $G = (V, E)$, where vertices V are primitive operations and edges E carry tensors between them. A *partition* of G into k shards is a division of V into disjoint subsets $V_1, \dots, V_k$. Each shard V_i induces a sub-computation SG_i (the subgraph restricted to V_i).

Definition 4.1 (Boundary and Boundary Width). *The boundary (cut-set) between shards V_i and V_j is $E_{cut}^{i,j} = \{(u, v) \in E \mid u \in V_i, v \in V_j\}$. The boundary data $T_{boundary}^{i,j}$ is the set of tensors on these edges. The boundary width is:*

$$W_B^{i,j} = \sum_{t \in T_{boundary}^{i,j}} \text{size}(t)$$

measured in scalar field elements. The total boundary width of a partition is $W_B = \sum_{i < j} W_B^{i,j}$.

A forward pass through G produces intermediate tensors $T_0, T_1, \dots, T_k$, where T_0 is the input and $T_i = \text{SG}_i(T_{i-1})$ for a sequential partition, or more generally, SG_i takes as input the tensors on all incoming cut edges.

Shard proofs and linking. The parallel proving phase generates a set of proofs $\{\pi_i\}_{i=1}^k$, where each π_i attests to the statement “ T_i is the correct output of computing SG_i on input T_{i-1} .” The choice of proof system per shard is independent: different shards may use different proof constructions.

Three security properties must hold for the composite proof to be sound:

1. *Shard validity:* each π_i is a valid proof for its sub-computation.
2. *Completeness:* the union of all shards spans the full computation graph. No shard is missing.
3. *Boundary consistency (linking):* the public output committed to in π_i is identical to the public input committed to in π_{i+1} , for all adjacent shards.

Linking cost. The cost of enforcing boundary consistency is a direct function of the boundary width W_B . Whether linking is implemented via recursive proof composition [19], accumulation schemes, or permutation arguments enforcing copy constraints across shard boundaries, the cost scales linearly with W_B : $C_{\text{link}} = O(W_B)$. For recursive formulations, the prover cost of π_{i+1} includes the cost of verifying π_i , which itself scales with the number of public inputs at the boundary.

Residual connections. In architectures with skip connections (ResNets, Transformers with residual streams), shard SG_{i+1} may depend not only on the output of SG_i but also on the output of some earlier shard SG_{i-j} . The boundary data must include these non-contiguous dependencies, and the linking proof must verify consistency across all incoming edges, not just the immediately preceding shard. This means the partition DAG has skip edges — edges

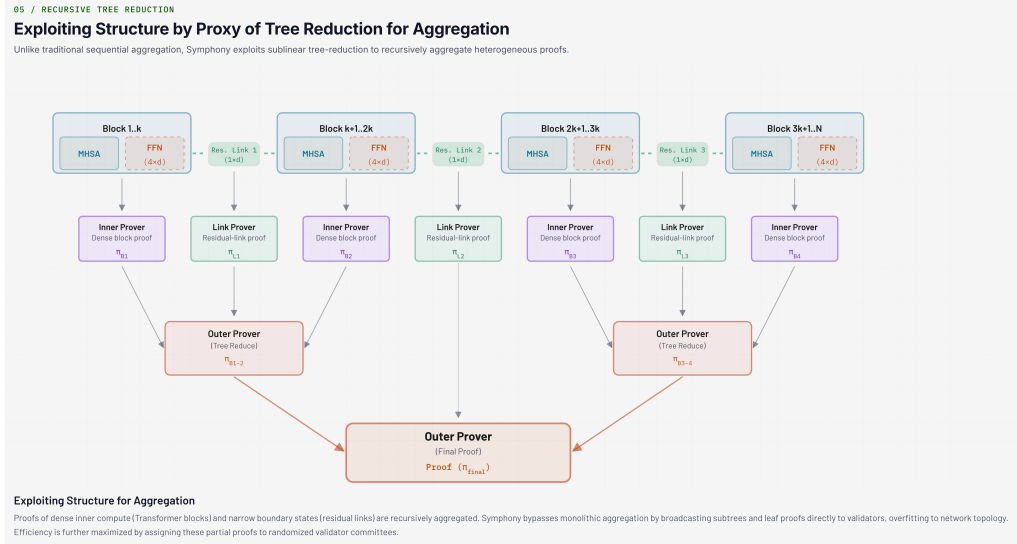


Figure 3: Structure-exploiting tree reduction for proof aggregation. Inner provers generate dense block proofs and residual-link proofs independently. Outer provers recursively aggregate subtrees, overfitting to network topology. Symphony bypasses monolithic aggregation by broadcasting subtrees and leaf proofs directly to randomized validator committees.

connecting non-adjacent shards, arising from residual connections in the underlying architecture. The boundary width W_B accounts for them: the boundary data at shard V_{i+1} includes tensors from both SG_i and SG_{i-j} .

Fat vs. narrow boundaries. Not all partition points are equal. A partition placed at a point where the data flow is narrow (few scalar elements crossing the boundary) incurs low linking cost. A partition placed inside an expansion layer (where the internal dimension is much larger than the input/output dimension) incurs high linking cost.

Optimal partitioning is a multi-objective problem: balance the computational complexity within each shard (to equalize prover workload) while minimizing W_B (to reduce linking overhead). For architectures with alternating expansion and contraction (the common pattern in neural networks), the narrow points between components are the natural partition boundaries.

No aggregation. Because shard proofs are verified by committees off-chain and are not stored on-chain (Section 4.3.5), the k shard proofs do not need to be recursively compressed into a single succinct proof. Each π_i is broadcast independently to its assigned committee. The only additional proof obligation beyond the individual shard proofs is boundary linking. This is a direct consequence of the prover-friendly design: aggregation is an optimization for on-chain verification cost, which does not apply here.

We refer to the graph-partition-based proving strategy described above — partitioning at narrow boundaries, exploiting compositional hierarchy and architectural symmetry, with independent shard proofs and boundary linking — as *structure-exploiting decomposition*.

Relationship to IVC. Incrementally verifiable computation (IVC) [79] decomposes by time step ($F = g^T$ for T iterations of step function g). Structure-exploiting decomposition decomposes by the structural units of the computational object (layers, heads, blocks), which may include both sequential components (depth) and parallel components (width). IVC requires a uniform step function; structure-exploiting decomposition accommodates heterogeneous subcomputations with varying resource profiles.

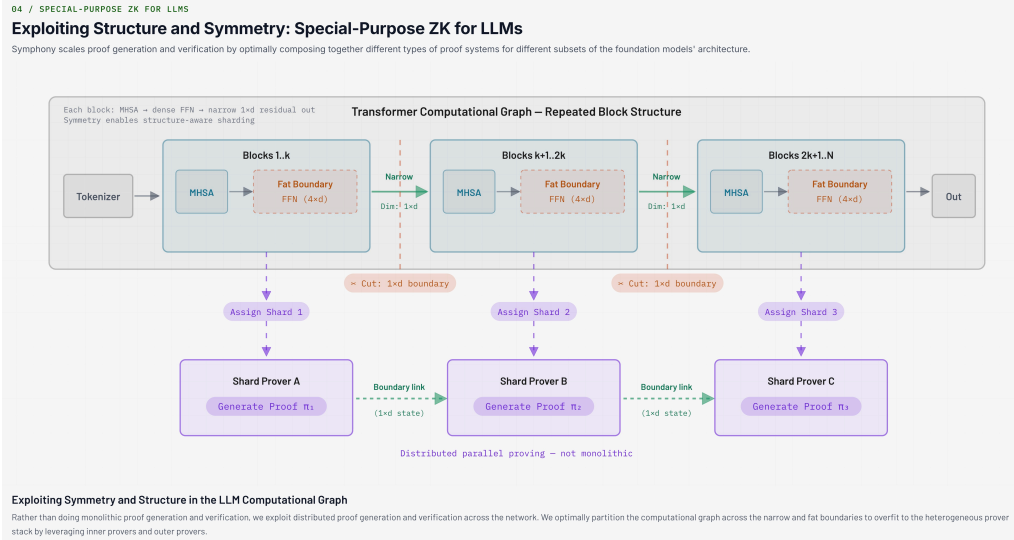


Figure 4: Structure-exploiting decomposition for transformer inference. Partition boundaries are placed at narrow inter-block interfaces (dimension d_{model}), avoiding fat boundaries inside FFN expansion layers (dimension $d_{\text{ffn}} \approx 4 \times d_{\text{model}}$). Each shard is proved independently; linking cost scales with boundary width.

4.2.3 Reference Decompositions: Transformers

The partition design principles above apply concretely to autoregressive transformer models. The key dimensions are: d_{model} (hidden dimension, e.g., 4096), d_{ffn} (FFN intermediate dimension, typically $4 \times d_{\text{model}}$), and s (sequence length). Tensors flowing between transformer blocks have shape (s, d_{model}) .

Narrow boundaries. The natural partition points are between transformer blocks, or between the multi-head self-attention (MHSA) sub-layer and the feed-forward network (FFN) sub-layer. At these points, the boundary tensor has dimension d_{model} , giving $W_B = s \cdot d_{\text{model}}$.

Fat boundaries. The FFN sub-layer projects from d_{model} to $d_{\text{ffn}} \approx 4 \cdot d_{\text{model}}$, applies a nonlinearity, then projects back to d_{model} . Partitioning *inside* the FFN expansion creates a boundary tensor of dimension d_{ffn} , increasing W_B by a factor of $\sim 4\times$ and correspondingly increasing the linking cost.

Partition strategy. Group consecutive transformer blocks into M partitions, cutting at the narrow inter-block boundaries. Within each partition, attention heads provide further parallelism (heads are independent up to the output projection). Residual connections within a partition are internal and do not affect W_B ; residual connections *across* partition boundaries add skip-edge tensors to the boundary data.

Other architectures. Convolutional networks decompose by stage (convolution-pooling groups), with feature maps as boundary data. Diffusion models decompose by timestep group, with the noisy image tensor at each boundary. Models exhibiting embarrassingly parallel structure (embedding lookups, random forests, mixture-of-experts routing) can be partitioned ahead of time without a forward pass. For arbitrary workloads, the developer provides a decomposition manifest specifying the sub-computation DAG, interface types, and partition assignment.

4.2.4 Verification Units

Each shard proof π_i (plus its boundary linking data) constitutes one *verification unit*. The total number of verification units for a computation is k (the number of shards). Each verification unit is assigned to a validator committee independently.

4.3 Distributed Verification via Committee Sampling

Full-network verification (every validator checks every proof) provides the strongest safety guarantee but is redundant in the sense that proof validity is a deterministic property. Committee-based verification introduces a different trust model: the SMR protocol no longer relies on every validator independently verifying every proof, but instead trusts that sampled committees with honest majorities will correctly report verification results. This reduces per-validator load but changes the safety guarantee from deterministic to statistical, parameterized by the committee compromise probability (Section 9.1.5).

4.3.1 Committee Assignment

Proof shards are assigned to validator subsets (committees) via deterministic sampling. The mechanism draws from two traditions:

- Algorand’s cryptographic sortition [43]: committee members self-select using a VRF, preventing an adaptive adversary from targeting committee members before they reveal themselves.
- Ethereum’s beacon chain committees [22]: deterministic assignment of validators to committees for attestation duties, where the assignment is public but the large validator set limits adaptive targeting.

Symphony’s committee assignment uses a verifiable random function seeded by the epoch’s randomness beacon: $(\text{epoch}, \text{shard_id}) \rightarrow \text{committee}$. **Note:** unlike Algorand’s sortition, Symphony’s deterministic assignment reveals committee membership before verification begins, creating an adaptive-corruption surface. If an adversary can corrupt or DoS specific validators after learning their committee assignment, the hypergeometric safety bounds (Section 9.1.5) do not apply. Mitigations include short epochs (limiting the corruption window) and requiring committees to be large enough that targeted corruption of a majority is impractical. A full treatment of adaptive adversaries is left to future work.

4.3.2 Specialization

Validators opt into verification duties by workload type and proof system type. ZKP verification does not require GPU hardware because proof verification is a CPU-bound operation involving field arithmetic and hash evaluations that runs on commodity hardware. Validators without any specialized hardware can verify both ZKPs and hardware attestations.

A general-purpose fallback ensures that every proof shard has a committee: validators that do not opt into any specialization are assigned to the fallback pool. Specialization enables verification key caching (a validator repeatedly verifying the same proof system type can cache verification parameters) and hardware-specific optimizations.

4.3.3 Quorum and Quorum-Skip

Each committee uses a simple majority quorum. Once a quorum of committee members has verified and attested to a proof shard, the shard is considered verified. The corresponding lattice dimension advances.

Validators that join the committee after quorum has been reached can *skip* verification of that shard—an optimization that reduces work for late-joining validators. This is analogous to Tendermint’s fast-sync [21]: a syncing validator can accept the quorum’s attestation rather than re-verifying from scratch. The security argument is identical: the quorum attestation is a certificate of the shard’s validity, backed by the committee’s stake. No safety is weakened; liveness improves for validators that are syncing or joining mid-epoch.

4.3.4 Security-Efficiency Tradeoff

The committee size k parameterizes the tradeoff between verification efficiency and safety. Smaller committees reduce the aggregate verification work (each proof shard is checked by k validators instead of n) but increase the committee compromise probability (Section 9.1.5).

The total verification work scales as:

$$\text{Total work} = M \times S \times k \times c_{\text{verify}}$$

where M is the number of workload partitions, S is the number of proof shards per partition, k is the committee size, and c_{verify} is the per-shard verification cost. For full-network verification, $k = n$; for committee-based verification, $k \ll n$, yielding a factor of n/k reduction in per-validator work on average.

The protocol selects k to achieve a target committee compromise probability (e.g., 10^{-9}) given the validator set size n and adversary bound f , using the hypergeometric bounds from Theorem 9.7.

4.3.5 Prover-Friendly Design Point

Committee-based off-chain verification changes which axis of the prover/verifier tradeoff is binding, relative to the on-chain verification model used by ZK rollups.

In ZK rollups, the proof is verified by an L1 smart contract. Every byte of proof costs gas; every verification cycle costs gas. This creates strong pressure to minimize proof size (constant-size SNARKs, recursive compression) and verification time (constant or polylog verification), even at the cost of slower proving.

Symphony’s verification model inverts this. Proofs are gossiped across the peer-to-peer layer to committee members who verify off-chain. Proof size is bounded by network bandwidth (hundreds of megabytes per second in practice), not by on-chain gas costs. Proofs are not stored on-chain. Verification cost is distributed across committee members on commodity hardware, not concentrated in a single EVM transaction. The binding constraint is *prover latency*: in the dual-finality model (Section 5), the interval between hardware attestation (milliseconds) and ZKP verification (minutes to hours) is the verification window during which safety depends on the TEE and the economic model. Every reduction in proving time shrinks this window and strengthens the system’s cryptographic safety posture.

Because proofs are not aggregated into a single on-chain artifact (Section 4.2.2), the system avoids the most expensive step in typical ZKP pipelines: recursive compression. Each shard proof is verified independently by its committee, so the prover never pays the cost of folding k proofs into one. This alone saves the majority of proving time. Additionally, because proof compactness is not a constraint (proofs are bounded by P2P bandwidth, not on-chain gas), the system is free to use proof constructions optimized for prover speed rather than succinctness. This includes GKR-based systems with $O(N)$ prover time (vs. $O(N \log N)$ for PLONK-style systems), constructions that trade proof size for fewer prover rounds, and schemes whose parallelization properties align with the shard decomposition. Verification cost per shard (c_{verify}) remains CPU-bound field arithmetic on commodity hardware, distributed across committee members.

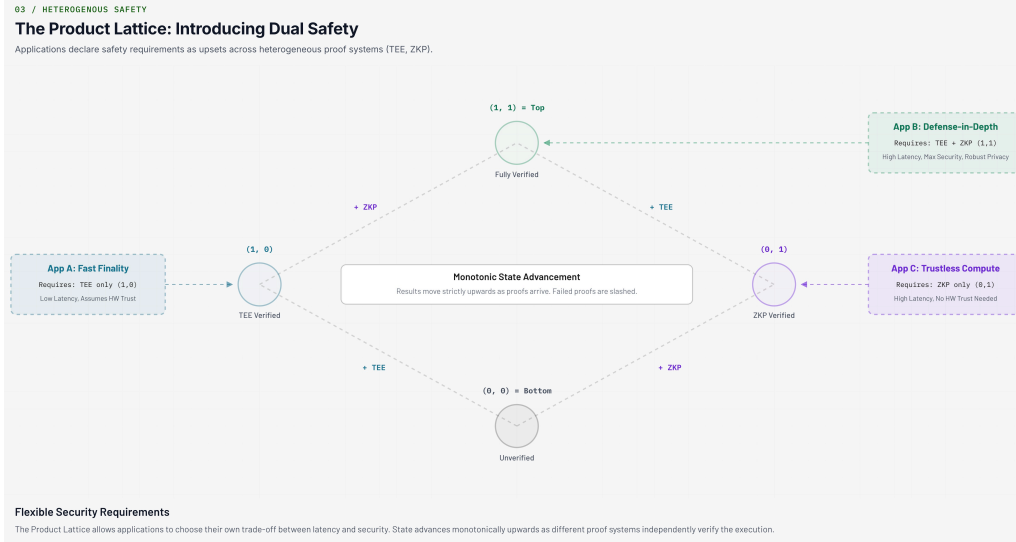


Figure 5: Product lattice over proof systems. Verification state advances monotonically as independent proof systems (e.g., TEE attestation, ZKP) provide attestations. Applications declare safety requirements as upsets (upward-closed regions) in the lattice; a result becomes accessible when its state enters the required upset.

5 Multi-System Verification Model

The verification state vector over proof systems, introduced in Section 2.4, is the data structure through which applications interact with delegated computation results. Verification states form a boolean hypercube $\{0, 1\}^m$ with componentwise ordering. This standard structure gives the paper a natural partial order (“at least this much verification”) and monotonicity properties. This section develops the operational semantics: how verification state evolves, how applications declare and consume safety requirements, how the no-rollback semantics works, and how proof system degradation is handled.

5.1 Lattice Structure and Verification State

Recall from Section 2.4 that the verification state of a delegated computation result o is:

$$\sigma(o) = (v_1, \dots, v_m) \in \{0, 1\}^m$$

where $v_i = 1$ indicates that proof system V_i has verified the result (committee quorum attained). The product lattice $(\{0, 1\}^m, \leq)$ with componentwise ordering has 2^m elements.

For $m = 2$ (hardware attestation V_1 and SNARK V_2), the lattice has four elements:

$$\perp = (0, 0) \quad (0, 1) \quad (1, 0) \quad \top = (1, 1)$$

and the Hasse diagram is a diamond. For $m = 3$, the lattice is a cube with 8 elements and 12 edges.

5.2 Soundness-Completeness-Latency Pareto Frontier

No single proof system dominates all others across all three dimensions of soundness, completeness, and latency. This is the fundamental reason for using multiple proof systems rather than selecting the “best” one.

Hardware attestations achieve high completeness (if the hardware is functioning, the attestation is always produced), low latency (milliseconds), but correlated soundness (a vendor-wide vulnerability compromises all attestations from that vendor simultaneously). They dominate the latency-completeness frontier but not the soundness frontier.

Cryptographic proofs (SNARKs, STARKs) achieve high soundness (negligible error under computational assumptions, independent across instances), but lower completeness (the prover may fail for very large computations) and high latency (minutes to hours for large workloads). They dominate the soundness frontier but not the latency frontier.

The product lattice exposes this tradeoff to applications. An application that needs fast finality can declare a safety requirement of $(1, 0)$ (hardware attestation sufficient). An application that needs cryptographic soundness can require $(0, 1)$ (SNARK sufficient) or $(1, 1)$ (both). The lattice is the mechanism through which applications navigate the Pareto frontier.

5.3 Failure Correlation Analysis

The soundness composition proposition (Theorem 2.7) requires independence of failure modes. In practice, independence must be assessed per proof system pair.

Independent failures. Hardware attestation (TDX) and cryptographic proof systems (SNARKs, STARKs) fail for unrelated reasons: the former via hardware compromise (physical attack, firmware vulnerability), the latter via cryptographic assumption failure. These failures are independent under any reasonable threat model.

Correlated failures. Multiple hardware attestations from the same vendor are correlated: a firmware vulnerability affects all instances simultaneously. The correlation structure is *per-vendor*: attestations from different vendors (Intel SGX, AMD SEV, ARM TrustZone) are approximately independent.

This analysis follows standard reliability theory [10]. Common-mode failure analysis identifies correlated failure channels and ensures that the lattice dimensions assigned to correlated systems are not treated as independent in the safety analysis.

Definition 5.1 (Failure Correlation Matrix). *For m proof systems, the failure correlation matrix $\rho \in [0, 1]^{m \times m}$ has entries ρ_{ij} denoting the correlation between failure events of systems V_i and V_j . Independence corresponds to $\rho_{ij} = 0$ for $i \neq j$.*

Applications requiring high combined soundness should specify proof systems from orthogonal dimensions in the lattice — those with $\rho_{ij} \approx 0$. The lattice structure makes this explicit: the soundness guarantee for a lattice position $\sigma = (v_1, \dots, v_m)$ depends on the correlation structure among the proof systems with $v_i = 1$.

5.4 Monotonic State Advancement

Verification state transitions are monotonically increasing in the lattice.

Invariant 5.2 (Monotonicity). *For any result o , the verification state $\sigma(o)$ can only advance: v_i can transition from 0 to 1, never from 1 to 0. No rollback of verification attestations occurs.*

This invariant holds because:

1. Each $v_i = 0 \rightarrow 1$ transition is guarded by a committee quorum attestation. Once a quorum of validators has attested, the attestation is recorded on-chain and is immutable (backed by the SMR protocol’s finality).

2. There is no mechanism for “un-verifying”: proof system verification is a one-way gate.

The lattice has two absorbing regions:

- **Top:** $\sigma(o) = \top = (1, \dots, 1)$. All proof systems have verified. This is the terminal state for a correctly executed computation.
- **Disputed:** a special status entered when proof systems disagree (one accepts, another rejects). If V_i has verified ($v_i = 1$) but V_j rejects (v_j cannot advance to 1 because the committee quorum rejected the proof), the result’s status transitions from **Settled** to **Disputed**. Disputed results are quarantined: no application can read them regardless of their lattice position. Formally, accessibility (Definition 2.6) is conditioned on both the lattice position and the result status: a result o is accessible to an application with requirement U if and only if $\sigma(o) \in U$ and $\text{status}(o) = \text{SETTLED}$.

5.5 Temporal Consistency

Different proof systems complete verification at different times. Hardware attestations complete in milliseconds; SNARK proofs may take minutes to hours. During this interval, different applications observe different lattice positions depending on their declared safety requirements.

Symphony provides three consistency guarantees, each grounded in the distributed database literature:

Monotonic reads [77]. Once an application observes a result o at lattice position σ , it never subsequently observes o at a position $\sigma' < \sigma$. This follows directly from the monotonicity invariant: the on-chain state only advances.

Snapshot isolation [13]. An application’s view of the result registry is consistent with a specific block height. Within a single transaction, the application sees a snapshot of all results at their lattice positions as of that block.

Bounded staleness [9]. The gap between the “latest” lattice position of a result (the position that would be observed if all pending verifications completed instantly) and the application-visible position is bounded by the latency of the slowest required proof system. For a result requiring verification by proof systems V_i with latencies ℓ_i , the bounded staleness is:

$$\delta_{\text{stale}}(o) = \max_{i \in \text{req}(U), v_i=0} \ell_i$$

This quantity decreases monotonically as proof systems complete. An application requiring only hardware attestation experiences staleness $\leq \ell_{\text{HW}} \approx$ milliseconds. An application requiring a SNARK proof experiences staleness $\leq \ell_{\text{ZKP}} \approx$ minutes to hours.

5.6 Irrevocable Execution with Economic Safety

Delegated computation results are not rolled back, even if a proof system later rejects the result. This is a deliberate design choice with consequences for the safety model.

Why no rollback. Rolling back a settled result would require reverting all downstream state transitions that consumed the result. In a blockchain with composable contracts, a single result may be consumed by hundreds of downstream transactions within seconds of settlement. Rollback would create a cascade of reversions, violating the finality expectations of users and applications that interacted with the post-settlement state. This is the same argument that motivates irrevocable transactions in the saga model [40]: compensating actions are preferable to rollback when the cost of rollback exceeds the cost of the incorrect state.

Economic safety argument. In the absence of rollback, safety is maintained economically. If a result y^* passes hardware attestation ($v_1 = 1$) but is later rejected by the SNARK proof ($v_2 = 0$), the result enters the **Disputed** state. The executor who submitted y^* is slashed: their bond b is distributed to affected parties (users who consumed y^* in downstream transactions, the protocol treasury, the committee that detected the dispute).

A rational executor does not submit incorrect results when $b \geq \text{MEV}(W, S)$. This is mechanism design [53, 72], not cryptographic safety. The assumptions are: rational adversary, correct bond sizing, live slashing mechanism.

Interaction with the lattice. The no-rollback semantics is reflected in the lattice: the **Disputed** state is absorbing (no transition out of it without protocol governance intervention). Applications that consumed the disputed result at a lower lattice position (e.g., based on hardware attestation only) accepted the risk encoded in their safety requirement. Applications that required the full lattice ($\top$) never consumed the result (it never reached $\top$) and are unaffected.

5.7 Application Safety Declarations

Applications declare their safety requirements as upsets in the lattice (Definition 2.6).

Declaration mechanism. A smart contract registers a safety requirement by calling a registry function:

`declareSafety( $U$ )` where $U \subseteq \{0, 1\}^m$ is an upset.

In practice, upsets are specified by their minimal elements (the antichain generating the upset), which is compact: for $m = 2$, the antichain has at most 2 elements.

Join propagation. When contract A calls contract B and B consumes a delegated computation result, the effective safety requirement is the join (least upper bound) of A 's and B 's requirements. This ensures that a caller never receives data verified at a level below its own requirement, even through transitive calls.

Explicit downcasting. A contract can explicitly lower its effective safety requirement for a specific read. This is logged on-chain (for auditability), scoped to the current transaction (does not affect the contract's declared requirement), and does not propagate to callers (the caller still sees the result through the lens of its own requirement). Downcasting is the mechanism by which applications opt into faster finality at reduced safety.

5.8 Proof System Degradation Protocol

When a proof system is compromised (e.g., a hardware vendor discloses a firmware vulnerability, or a cryptographic assumption is broken), the protocol enters a degraded mode for that system.

Detection. Proof system compromise is detected externally (vendor disclosure, academic publication) and reported via a governance transaction. The protocol does not attempt automatic detection of proof system compromise; this is a deliberate limitation (automatic detection of, e.g., a broken cryptographic assumption is an open problem).

Response.

1. The compromised proof system V_j is marked as **Degraded** in the protocol state.

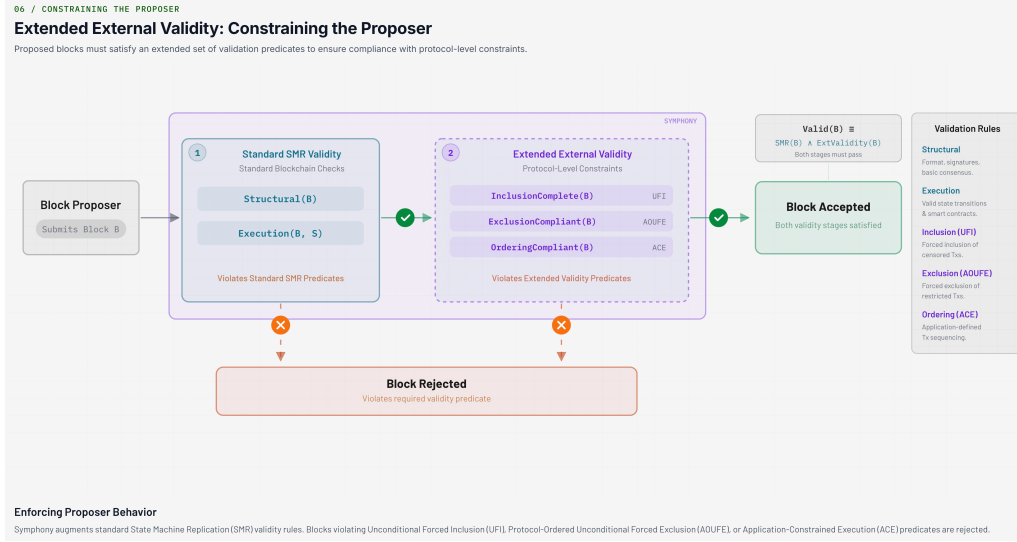


Figure 6: Extended external validity. Symphony augments standard SMR validity rules with three additional conjuncts—inclusion, exclusion, and ordering compliance—each enforced deterministically by every validator. Blocks violating any conjunct are rejected.

2. Applications that required V_j (i.e., their upset includes a dimension where $v_j = 1$) are notified: their effective safety requirement can no longer be met by existing proofs. They must either:
 - Accept a reduced safety level (explicitly downcast past the degraded dimension).
 - Wait for re-verification under a replacement proof system (if one is added).
 - Trigger a compensating action.
3. Pending results verified only by the degraded system (and no other) are flagged for quarantine via exclusion guarantees: transactions consuming these results are excluded until the results are re-verified.
4. New proofs under the degraded system are not accepted until the degradation is resolved.

This generalizes certificate revocation [52] and key compromise response to the proof system setting.

6 Extended External Validity

Section 3 addressed the first facet of execution-aware SMR: handling expensive and randomized workloads via delegated execution with verifiable computation. This section formalizes the second facet: protocol-enforced guarantees from proposers to users and applications. Extended external validity is the enforcement mechanism.

External validity, introduced by Cachin, Kursawe, and Shoup [24], is a property of SMR protocols that allows the validity condition on proposed values to reference application-specific predicates. In standard BFT protocols, the validity condition checks structural properties: well-formed blocks, valid signatures, correct state root. External validity extends this with predicates that reference the current state.

Symphony uses extended external validity as the single enforcement mechanism for all guarantees. Each guarantee from Section 2.6 adds a conjunct to the block validity function. The

proposer retains only the residual power not constrained by these conjuncts. This section defines the extended validity function, proves that predictable validity is necessary, formalizes the efficiency of predicate evaluation, and derives the censorship resistance / liveness tradeoff.

6.1 Extended Block Validity

Definition 6.1 (Extended Block Validity). *A block B proposed for state S is valid if and only if all of the following conjuncts hold:*

$$\text{Valid}(B, S) = \text{Structural}(B) \tag{1}$$

$$\wedge \text{Execution}(B, S) \tag{2}$$

$$\wedge \text{InclusionComplete}(B, S) \tag{3}$$

$$\wedge \text{ExclusionCompliant}(B, S) \tag{4}$$

$$\wedge \text{OrderingCompliant}(B, S) \tag{5}$$

The first two conjuncts are standard in any BFT protocol:

- $\text{Structural}(B)$: the block is well-formed, all transaction signatures are valid, the parent hash is correct, and the proposer is the designated leader for this round.
- $\text{Execution}(B, S)$: the proposed post-state S' is the correct result of applying the block's transactions to S . For replicated transactions, every correct validator re-executes and checks. For delegated transactions, correctness of the async commit/settlement transitions is verified against the protocol rules (Section 2.7).

The final three conjuncts are Symphony's extensions:

- $\text{InclusionComplete}(B, S)$: for every active inclusion trigger $(P_i, T_i, \text{TTL}_i) \in \mathcal{I}$ such that $P_i(S) = 1$, the transaction T_i (or a transaction satisfying the trigger's specification) appears in B . Section 7.1 defines the semantics.
- $\text{ExclusionCompliant}(B, S)$: for every active exclusion trigger $(Q_j, \text{TTL}_j) \in \mathcal{E}$, no transaction $T \in B$ satisfies $Q_j(S, T) = 1$. Section 7.2 defines the semantics.
- $\text{OrderingCompliant}(B, S)$: for every contract C with a registered ordering policy $(\pi_C, \{\tau_i\})$, the subsequence of transactions in B that target C appears in the order specified by $\pi_C(\text{TxSet}_C, S)$ and $\tau_i(\text{TxSet}_C, S)$. Section 8 defines the semantics.

6.2 Power Guarantees as Validity Conjunct

All three guarantees follow the same structure. Each is implemented as a conjunct in Definition 6.1:

1. A beneficiary (user, application, or the protocol itself) registers a guarantee by publishing a predicate on-chain. This registers a trigger in $\mathcal{I}$, $\mathcal{E}$, or $\{(\pi_C, \{\tau_i\})\}$.
2. The predicate becomes a conjunct of the block validity function.
3. Every block proposed by a correct proposer must satisfy all active conjuncts.
4. Every correct validator independently evaluates all active conjuncts and rejects blocks that violate any. This deterministic evaluation is the default and the basis for Symphony's BFT safety argument.

5. The proposer retains only the *residual* discretion: the freedom to act that is not constrained by active transfers.

In the limit, if every possible proposer action is constrained by some transfer, the proposer becomes a deterministic automaton: the block’s contents are uniquely determined by the state and the active predicates [87]. This extreme is undesirable (it eliminates the proposer’s ability to perform discretionary optimization), but it illustrates that proposer discretion lies on a continuum.

6.3 Predictable Validity Requirement

Theorem 2.9 (Section 2.5) establishes that extended external validity with current-state predicates requires predictable validity: when a proposer constructs their block, they can exactly compute the state resulting from confirmation of their block. We restate the implication in terms of system design.

Corollary 6.2 (Protocol Compatibility). *Symphony’s extended external validity is compatible with SMR protocols that provide predictable validity with staleness 0, and partially compatible with protocols that provide predictable validity over a committed prefix with bounded staleness $\delta > 0$.*

For protocols with staleness $\delta > 0$ (e.g., DAG-based protocols where the committed prefix lags the latest state), predicates must be modified to account for staleness: a predicate $P(S)$ becomes $P_\delta(S) = P(S_{\text{committed}})$, where $S_{\text{committed}}$ is the state corresponding to the last committed prefix of the ledger. The staleness δ is the number of uncommitted blocks (or rounds) between $S_{\text{committed}}$ and the current tip. Validators evaluate P against the same $S_{\text{committed}}$, which is determined by SMR, avoiding the validator-disagreement risk inherent in existential quantification over arbitrary past states.

In practice, any linear-commit BFT protocol (e.g., CometBFT [21], HotStuff [83], MonadBFT [63], Alpenglow [60], Simplex [61], Minimmit [62]) provides staleness 0.

6.4 Censorship Resistance / Liveness Tradeoff

This section formalizes the fundamental tradeoff between censorship resistance (enforced through inclusion guarantees) and liveness (the protocol’s ability to produce valid blocks in a timely manner).

6.4.1 Setup

Let k be the number of active guarantees (inclusion triggers, exclusion triggers, and ordering policies). Let Δ_B be the block construction time budget: the maximum time a correct proposer has to construct a valid block before the round times out and the protocol moves to the next proposer.

Each active transfer imposes two costs on the proposer:

1. *Evaluation cost*: the proposer must evaluate the predicate $P_i(S)$ or $Q_j(S, T)$ for each relevant transaction. Let c_i be the computational cost of evaluating the i -th predicate.
2. *Constraint cost*: each satisfied predicate constrains the set of valid blocks. Let $\mathcal{B}_i(S)$ be the set of blocks valid under the i -th conjunct. The set of valid blocks under all conjuncts is $\mathcal{B}(S) = \bigcap_{i=1}^k \mathcal{B}_i(S)$.

6.4.2 Block Construction Feasibility

Definition 6.3 (Feasibility). *A state S with active transfers $\{T_1, \dots, T_k\}$ is feasible if $\mathcal{B}(S) \neq \emptyset$: there exists at least one valid block.*

Proposition 6.4 (Infeasibility from Conflicting Guarantees). *There exist sets of active guarantees that render a state infeasible. In particular, if an inclusion trigger requires inclusion of transaction T and an exclusion trigger requires exclusion of T , and both predicates are satisfied, then $\mathcal{B}(S) = \emptyset$.*

Proof. The inclusion conjunct requires $T \in B$, and the exclusion conjunct requires $T \notin B$. No block can satisfy both simultaneously. $\square$

Conflict detection is a satisfiability problem over the predicate set, formally related to constraint satisfaction [78] and the consistency of access control policies [49]. The protocol addresses this through conflict detection at registration time (Section 7.1).

6.4.3 Liveness under Power Guarantees

Adding guarantees increases the proposer’s critical-path work and constrains the set of valid blocks, creating a tension between censorship resistance and liveness.

Evaluation cost. Each active predicate adds evaluation work to block construction. With k active predicates and per-predicate costs $c_1, \dots, c_k$, the total work is $\sum_{i=1}^k c_i$. Predicate evaluation is embarrassingly parallel: inclusion predicates ($P_i : \mathcal{S} \rightarrow \{0, 1\}$) are independent of each other, exclusion predicates ($Q_j : \mathcal{S} \times \mathcal{T} \rightarrow \{0, 1\}$) are independent per-transaction, and ordering policies $\{\pi_C\}$ are independent per-contract. With P parallel evaluation slots, the wall-clock cost is $\sum_{i=1}^k c_i / P$. The protocol cap $k_{\max}$ bounds total work: $\sum_{i=1}^{k_{\max}} c_i \leq P \cdot \alpha \cdot \Delta_B$ for configurable $\alpha < 1$ and block construction budget Δ_B . The proposer faces one serial dependency: it must know all predicate outcomes before finalizing the block. Validators check a fixed block against predicates with no inter-predicate dependency, enabling full parallelism.

Feasibility. Each active predicate constrains the set of valid blocks: $\mathcal{B}(S) = \bigcap_{i=1}^k \mathcal{B}_i(S)$. Adding a predicate can only shrink this intersection. In the extreme, conflicting predicates make $\mathcal{B}(S) = \emptyset$ and no valid block exists (Proposition 6.4). Conflict detection at registration time (Section 7.3) bounds this risk.

Monotonicity. The magnitude of this tension is monotone in k : more active predicates mean more evaluation work and a more constrained valid-block set. Both effects are individually monotone (adding a predicate never reduces evaluation cost or enlarges the valid-block set). Parallelism and the protocol cap reduce the cost but do not eliminate the tension, which is inherent in any system that adds validity constraints.

Protocol mitigation. Symphony bounds the tradeoff through three mechanisms:

1. *Protocol cap on active predicates:* $k_{\max}$ limits the number of simultaneously active inclusion/exclusion triggers.
2. *Economic filtering:* registering a trigger requires a deposit that is slashed if the predicate is malformed or excessively expensive to evaluate.
3. *Conflict detection:* overlapping inclusion and exclusion predicates are checked at registration time. Conflicting registrations are rejected unless a resolution policy is declared. A natural relaxation (future work) is to resolve conflicts via an auction over predicate slots, pricing slot contention rather than rejecting it outright.

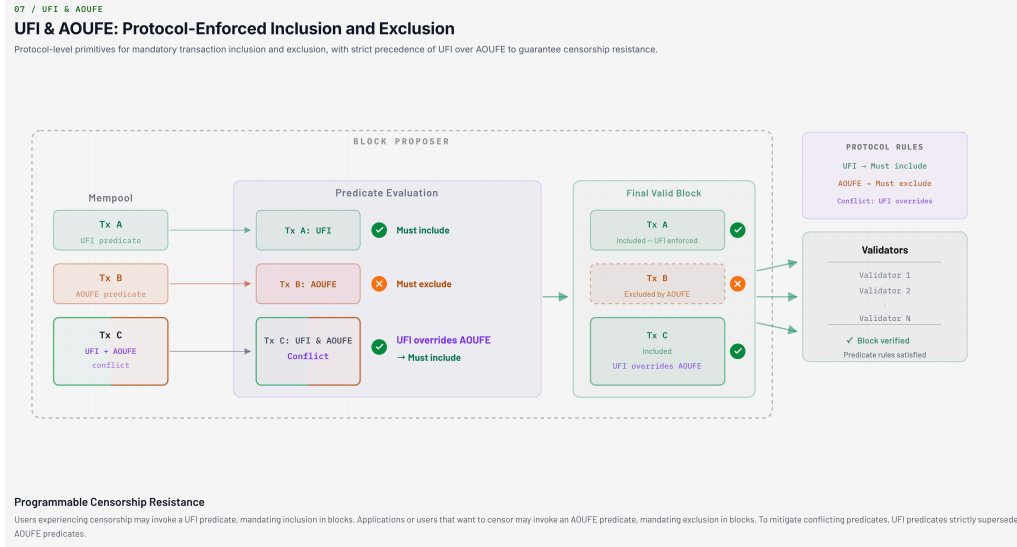


Figure 7: Inclusion and exclusion enforcement pipeline. The proposer evaluates mempool transactions against active inclusion triggers (UFI) and exclusion triggers (AOUE). Inclusion mandates take strict precedence over exclusion to prevent censorship vectors.

6.5 Predicate Evaluation

Evaluating all active predicates on every block proposal is the critical-path cost of extended external validity.

6.5.1 Default: Full Deterministic Evaluation

In the default mode, every correct validator independently evaluates every active predicate. The proposer includes an *evaluation certificate*, a summary of which predicates are satisfied and which transactions are mandated or excluded. Validators treat this as advisory and perform full re-evaluation. This is the mode under which Symphony’s BFT safety proofs hold: all correct validators agree on validity because they all evaluate the same deterministic predicates over the same predictably valid state.

The total work is $O(n_{\text{trig}})$ per validator per block, where n_{trig} is the number of active predicates. Since predicates are independent, this work is parallelizable across P cores, yielding wall-clock cost $O(n_{\text{trig}}/P)$. The protocol cap k_{max} (Section 6.4.3) bounds the total work.

Dependency tracking. Each predicate declares the set of state slots it reads. When a block modifies state slot s , only predicates with s in their dependency set need re-evaluation. This reduces the amortized cost to $O(\text{changed_slots} \times \text{dependent_predicates})$.

7 Inclusion and Exclusion Guarantees

This section instantiates two of the three guarantees from Section 2.6: inclusion and exclusion. These guarantees are enforced in-protocol via the `InclusionComplete` and `ExclusionCompliant` conjuncts of the extended block validity function (Definition 6.1).

7.1 Inclusion Guarantees

Definition 7.1. An inclusion trigger is a triple (P, T, TTL) registered in the chain state:

- $P : S \rightarrow \{0, 1\}$ is a state predicate.

- T is a transaction.
- $TTL \in \mathbb{N}$ is the time-to-live in blocks.

Definition 7.2 (Enforcement of inclusion guarantees). *A block B proposed for state S satisfies the **InclusionComplete** conjunct if and only if: for every active trigger $(P_i, T_i, TTL_i) \in \mathcal{I}$ such that $P_i(S) = 1$ and the trigger has not expired, T_i appears in B .*

A block that omits a triggered transaction is invalid. Every correct validator rejects it. The proposer cannot censor the transaction without producing an invalid block that the consensus protocol rejects. This is the strongest form of censorship resistance available: consensus-level block rejection, not economic incentive or social norm. The cost of evaluating inclusion predicates in the block construction path, the existence of valid blocks under active triggers, and the efficiency of finding a valid block (even without revenue maximization) are analyzed in Section 6.4.3.

7.1.1 Comparison to Existing Mechanisms

Inclusion lists (EIP-7547) [35]. Ethereum’s inclusion lists provide validity-level enforcement with a one-block delay: the next proposer’s block is invalid if the current proposer’s inclusion list is not satisfied. Inclusion guarantees differ in two respects. First, inclusion guarantees provide same-block enforcement for every transaction after the initial registration: once a trigger is registered, the proposer’s own block is invalid if the triggered transaction is absent, with no reliance on the next proposer’s compliance. Second, inclusion guarantees are non-interactive after registration: The user registers a trigger once and does not need to be online or submit new lists each slot for the CR guarantee to hold. Inclusion lists are just-in-time (reactive, per-slot); inclusion triggers are registered in advance (proactive, persistent until TTL expiry). The initial registration transaction itself has the same CR properties as any other transaction—it depends on the underlying censorship resistance of the base protocol. The distinction is in what happens after registration: Inclusion guarantees provide persistent, non-interactive censorship-resistance for every subsequent triggering; inclusion lists require active, per-slot participation.

Chainlink Keepers. External actors are incentivized to execute maintenance transactions. There is no protocol guarantee of inclusion, merely a reliance on economic incentives and the willingness of keepers to submit transactions. Inclusion guarantees provide protocol-level guarantees independent of external actor incentives.

SQL triggers and active databases [30, 70]. The closest conceptual ancestor. Event-Condition-Action (ECA) rules in active databases fire automatically when a condition holds. Symphony’s inclusion triggers are ECA rules enshrined at the consensus level: the “event” is a state change, the “condition” is the predicate P , and the “action” is forced inclusion of T . The trust model differs fundamentally: in an active database, the DBMS is trusted; in a blockchain, the proposer is untrusted, and enforcement must be Byzantine fault-tolerant.

7.2 Exclusion Guarantees

Exclusion guarantees are the converse of inclusion guarantees.

Definition 7.3. *An exclusion trigger registered by contract C is a pair (Q, TTL) where:*

- $Q : \mathcal{S} \times \mathcal{T} \rightarrow \{0, 1\}$ is a predicate over state and transactions.
- $TTL \in \mathbb{N}$ is the time-to-live in blocks.

Scope restriction. Q may only evaluate to 1 for transactions that target the registering contract C . An exclusion trigger cannot exclude transactions that do not interact with C .

Definition 7.4 (Enforcement of exclusion guarantees). *A block B proposed for state S satisfies the *ExclusionCompliant* conjunct if and only if: for every active trigger $(Q_j, TTL_j) \in \mathcal{E}$ and for every transaction $T \in B$, $Q_j(S, T) = 0$.*

Equivalently: no transaction in B satisfies any active exclusion predicate. A block containing such a transaction is invalid. As with inclusion guarantees, the cost of evaluating exclusion predicates, block building complexity under active exclusion triggers, and the existence of computationally efficient strategies for finding a valid block are analyzed in Section 6.4.3.

The simplest user-defined exclusion guarantee would be attached to a single transaction (represented as Q only ever being equal to 1 when its second argument is a specific transaction). For example, a user might want to include an exclusion predicate dictating that a DEX trade won't be included if the price is below a threshold. Note that slippage-based transactions *can* be included onchain even though they revert, whereas here, the transaction is not includable and therefore does not pay any gas. Exclusion guarantees can be more general than per-transaction predicates, and can allow applications or the protocol itself to specify exclusion triggers over sets of transactions.

Comparison to inclusion predicates. Inclusion predicates have signature $P : \mathcal{S} \rightarrow \{0, 1\}$ (state-only); the transaction to include is pre-specified at registration time. Exclusion predicates have signature $Q : \mathcal{S} \times \mathcal{T} \rightarrow \{0, 1\}$ (state and transaction); they must inspect each candidate transaction because the set of transactions to exclude is not known in advance. This asymmetry is fundamental: Inclusion triggers know *what* to include and check *when*; exclusion triggers check *whether* each incoming transaction should be blocked, given the current state.

Limitations of the single-transaction signature. $Q(S, T)$ evaluates one transaction at a time. Multi-transaction relational constraints (e.g., “transaction T_1 must not appear within k positions of T_2 ”) would require extending the predicate domain to reference the full block transaction set. The current signature cannot express relative-position constraints between transactions.

7.2.1 Use Cases

Exclusion guarantees enable a class of application-level safety controls that are not available in any existing blockchain protocol:

Application circuit breakers. A DeFi contract registers an exclusion trigger: “exclude all transactions targeting me when the oracle staleness exceeds 60 blocks.” The predicate is $Q(S, T) = (S.\text{blockHeight} - S.\text{oracleLastUpdated}[C] > 60)$, where C is the registering contract (the scope restriction ensures Q is only evaluated for transactions targeting C). During oracle downtime, the contract is automatically protected from stale-data trades. Staleness is measured in blocks rather than wall-clock time, since block height is the only quantity with a strongly consistent view across all validators.

Anti-sandwich constraints. Preventing sandwich attacks (“transaction T_1 must not appear within k positions of T_2 ”) requires relative-position constraints between transactions, which exceeds the single-transaction signature $Q(S, T)$. This use case would require extending exclusion triggers to predicates over the block's transaction set, or is better addressed by ordering constraints (Section 8) or the lazy evaluation pattern.

Quarantine during proof system degradation. When a proof system enters degraded mode (Section 5.8), exclusion triggers automatically exclude transactions that consume results verified only by the degraded system. This is a protocol-level quarantine that prevents downstream contamination from potentially-unsound results.

Protocol-level invariant enforcement. The protocol itself can register exclusion triggers to enforce safety invariants: exclude transactions that would create more than N pending delegated computations (preventing resource exhaustion), exclude transactions from sanctioned addresses (compliance), or exclude transactions that would exceed a contract’s per-block gas limit (DoS protection).

7.2.2 Comparison to Existing Mechanisms

Transaction filtering (OFAC compliance in Ethereum). Some Ethereum builders voluntarily filter transactions from sanctioned addresses. This is not enforced at the protocol level; non-filtering builders can include any transaction. Exclusion guarantees are a protocol-enforced property, independent of builder choice.

MEV protection services (Flashbots Protect) [38]. Private transaction channels provide two properties: (a) information hiding (the user’s trade intent is not visible in the public mempool, preventing front-running based on observed intent) and (b) builder commitment to not sandwich protected transactions. Exclusion guarantees address (b) at the protocol level: exclusion predicates can prevent sandwich-pattern transactions from entering the block, without trusting the builder. Exclusion guarantees do not address (a): the user’s transaction is still visible in the public mempool. Information hiding requires encrypted mempools [74] or private relay channels, which are complementary to exclusion guarantees rather than replaced by them.

Rate limiting and access control. Smart contracts can implement their own rate limiting (e.g., “revert if called more than N times per block”), but this consumes gas and block space even for reverted transactions. Current approaches to on-chain access control on L2s use optimistic execution patterns that similarly consume resources on failure [68]. Exclusion guarantees prevent the transaction from entering the block in the first place, saving gas and block space. The exclusion check happens during block construction, before execution, so excluded transactions impose no execution cost on the network.

7.3 Interaction between inclusion and exclusion

Inclusion and exclusion predicates can conflict: an inclusion trigger may mandate inclusion of a transaction that an exclusion trigger mandates exclusion of. Proposition 6.4 (Section 6.4.3) shows that conflicting triggers can render block construction infeasible.

7.3.1 Conflict Detection

At registration time, the protocol checks for potential conflicts between the new trigger and all active triggers. The check is conservative: if the predicate domains of an inclusion trigger and an exclusion trigger overlap (they reference the same transaction specification or the same contract), a potential conflict is flagged.

Static analysis. For predicates with declared dependency sets, the protocol checks whether the state-slot dependencies overlap. If the inclusion predicate P and the exclusion predicate Q read disjoint state slots, they cannot conflict regardless of the state.

Registration-time evaluation. The protocol evaluates both predicates at the current state. If $P(S) = 1$ and $Q(S, T) = 1$ for the inclusion-specified transaction T , an immediate conflict exists and the registration is rejected.

Limitations. Static analysis is sound but not complete: it may reject non-conflicting registrations (false positives) but will not accept conflicting ones (no false negatives). Full conflict detection (whether there exists *any* future state S' such that $P(S') = 1$ and $Q(S', T) = 1$) is undecidable in general (by reduction from satisfiability of state predicates over a Turing-complete state machine). This is formally related to the safety analysis of access control policies [49], where determining whether a protection system can reach an unsafe state is undecidable in the general case.

7.3.2 Runtime Conflict Resolution

If a conflict arises at runtime (due to a state change that simultaneously triggers inclusion and exclusion predicates on the same transaction), the protocol applies a resolution policy.

1. **Default: inclusion overrides exclusion.** The transaction is included. An inclusion trigger is enshrined in the finalized chain state: the chain has already committed to including T_i when $P_i(S) = 1$. This is a consensus-level mandate, not a discretionary inclusion. Exclusion guarantees, by contrast, constrain the proposer’s discretionary inclusion power over mempool transactions. An inclusion-mandated transaction is not in the proposer’s discretionary set, so exclusion guarantees do not apply to it. The conflicting exclusion trigger remains active for transactions matching its pattern without an active inclusion trigger.
2. **Configurable.** The resolution policy can be overridden at registration time: “exclusion overrides inclusion” (prioritize exclusion in specific safety-critical contexts, e.g., quarantine during proof system degradation) or “reject block” (force the proposer to wait for a non-conflicting state). The choice is application-specific and must be declared when the conflicting trigger is registered.

Mechanism-based resolution (future work). The precedence rules above are static: they are declared at registration time and applied deterministically. The alternative is via a mechanism design framework, whether an auction or a contractual agreement between a prespecified set of actors on how to handle conflict resolution.

Why inclusion takes precedence by default. The purpose of exclusion guarantees is to prevent the proposer from including harmful transactions. But a transaction mandated by an inclusion guarantee was explicitly registered by a user or the protocol as a consensus-level obligation. Allowing an exclusion guarantee to override it would create a censorship vector: a malicious application developer that controls the exclusion constraints *on a contract called by* the victim transaction could register an exclusion predicate overlapping with a victim’s inclusion trigger to neutralize forced inclusion indefinitely. With inclusion-overrides-exclusion as the default, this attack is not possible. Applications that genuinely need exclusion to override inclusion (e.g., emergency quarantine) must opt into that policy explicitly at registration time, accepting the censorship risk.

7.4 Integration with the Verification Lattice

Inclusion and exclusion guarantees interact with the product lattice in two directions:

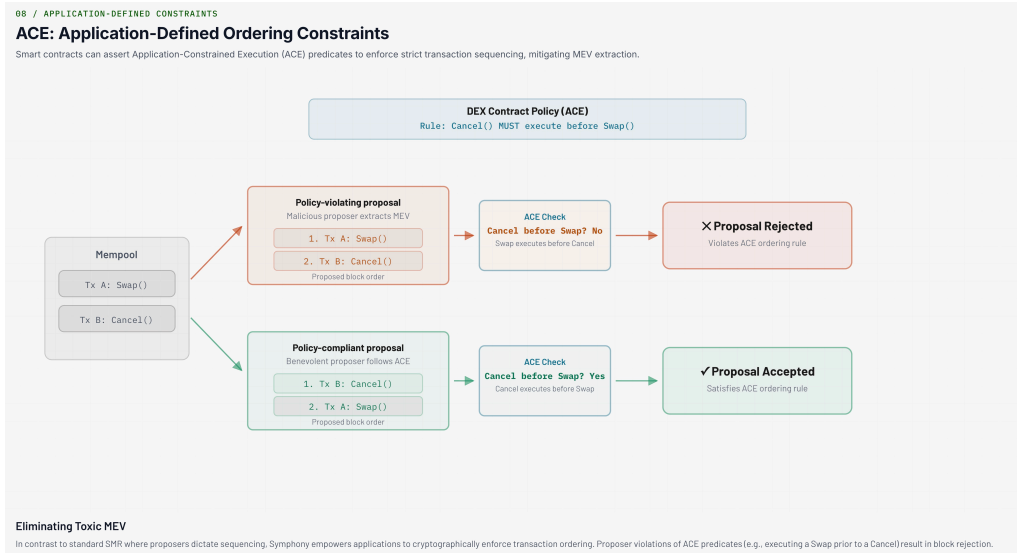


Figure 8: Application-defined ordering (ACE). A smart contract declares an ordering policy over recognized call sequences. A malicious proposer submitting transactions in a policy-violating order is deterministically rejected; an honest proposer complying with the policy is accepted.

Lattice promotions as inclusion triggers. When a delegated computation result advances to a new lattice position (e.g., SNARK verification completes, advancing from $(1, 0)$ to $(1, 1)$), an inclusion trigger can fire: “when result R reaches lattice position $(1, 1)$, include transaction T that consumes R .” This chains verification completion with downstream execution.

Proof system degradation as exclusion triggers. When a proof system enters degraded mode, exclusion triggers automatically activate: “exclude transactions that consume results verified only by the degraded system.” This is a state-dependent exclusion guarantee whose predicate references the proof system’s operational status in the chain state.

These interactions demonstrate the composability of guarantees: inclusion and exclusion can be applied concurrently and in support of other system components (the lattice, the degradation protocol) through the same predicate-based mechanism.

8 Application-Defined Ordering Guarantees

The third group of guarantees from Section 2.6 transfers the proposer’s sequencing power to individual applications, allowing each application contract to declare its own ordering policy.

8.1 Transaction Ordering as Application-Layer Authority

In standard blockchain protocols, the proposer has unilateral authority over transaction ordering within a block. This authority is the root of ordering-dependent MEV: sandwich attacks, frontrunning, and backrunning all exploit the proposer’s (or builder’s) ability to choose the sequence in which transactions execute.

Application-controlled execution (ACE) has been deployed in practice. Hyperliquid enforces cancel prioritization at the L1 level: within each block, cancel orders must precede take orders, improving spreads by protecting market makers from stale-quote extraction. This is the most notable deployed instance of ACE and demonstrates that application-specific ordering yields measurable trading improvements [7].

More broadly, ordering guarantees can be implemented at multiple levels [7]: as an app-chain (Hyperliquid), via on-chain batching with delayed execution (AMQs on Solana), via off-chain

sequencing delegated to specialized parties (Sorella), through voluntary proposer commitments (PEPC), or enshrined in L1 consensus with mandatory validator participation. Symphony implements the last of these: ordering constraints are conjuncts of the block validity function, checked by every validator.

Symphony’s ordering guarantees differ from prior work on ordering-fairness in three respects:

1. **Per-contract, not global.** Each contract declares its own ordering policy. An AMM contract may order transactions by price impact. A lending contract may order liquidations by collateral ratio.
2. **Semantic, not temporal.** Ordering policies are functions of transaction content and chain state, not arrival times. This enables application-specific ordering logic that temporal rules cannot express.
3. **Simultaneous heterogeneous policies.** Multiple contracts with different ordering policies coexist in the same block. Transactions targeting contract A are ordered by A ’s policy; transactions targeting contract B are ordered by B ’s policy; transactions targeting neither are ordered at the proposer’s discretion.

8.2 Formal Definition

Let $\mathcal{F}_C = \{f_1, \dots, f_n\}$ be the set of external function signatures of contract C , and let $\mathcal{F}_C^*$ denote the set of all finite sequences over $\mathcal{F}_C$ (including repetitions). A single transaction targeting C may invoke any number of functions on C , any number of times, in any order. The *call sequence* $\sigma(T)$ of a transaction T on contract C is the ordered list of function invocations on C observed in T ’s execution trace.

Definition 8.1 (Ordering Policy). *An ordering policy for contract C consists of:*

- *A finite ordered list of recognized call sequences:*

$$\pi_C = [s_1, s_2, \dots, s_k] \quad \text{where each } s_i \in \mathcal{F}_C^*$$

The position i is the sequence’s priority (lower index = higher priority). The policy has finite support: only finitely many call sequences are recognized.

- *An optional tiebreaker function per recognized sequence: $\tau_i : \mathcal{S} \times \mathcal{T} \times \mathcal{T} \rightarrow \{-1, 0, 1\}$, evaluated on finalized state S , that orders transactions matching the same s_i (or none of the s_i). If no τ_i is declared, the proposer retains discretion within that priority level.*

All other sequences in $\mathcal{F}_C^*$ are *unrecognized* and are automatically assigned the lowest priority: every transaction with a recognized call sequence must precede every transaction with an unrecognized call sequence in the block. Among transactions with an unrecognized call sequence, the proposer retains full ordering discretion. This avoids requiring the contract to enumerate all possible call sequences: the contract specifies only the patterns it cares about, and everything else is deprioritized by default. This choice gives extra confidence to application designers and users who know that any searcher trying to use the contract in unexpected ways will always be sequenced after the “expected” user transactions.

The tiebreaker τ_i is evaluated on the finalized state (the pre-state of the current block), so the intra-priority ordering is deterministic and known before block construction begins. This is how state-dependent ordering (e.g., sorting liquidations by collateral ratio) is expressed within the static call-sequence framework.

Definition 8.2 (Enforcement of ordering guarantees). *A block B proposed for state S satisfies the **OrderingCompliant** conjunct if and only if: for every contract C with a registered ordering policy $(\pi_C, \{\tau_i\})$ and for every pair of transactions $T_i, T_j \in B$ targeting C :*

- If both $\sigma(T_i) = s_a$ and $\sigma(T_j) = s_b$ are recognized with $a < b$, then T_i appears before T_j in B .
- If $\sigma(T_i)$ is recognized and $\sigma(T_j)$ is unrecognized, then T_i appears before T_j in B .
- If both $\sigma(T_i) = \sigma(T_j) = s_a$ (same recognized sequence) and τ_a is declared, then T_i and T_j must appear in the order specified by $\tau_a(S, T_i, T_j)$.
- If both $\sigma(T_i)$ and $\sigma(T_j)$ are unrecognized, no ordering constraint applies.

Recognized transactions are ordered by their priority in π_C and always precede unrecognized transactions. Within the same priority level, the tiebreaker τ_i (if declared) determines ordering based on finalized state. Among unrecognized transactions, the proposer retains discretion.

Note that the tiebreaker semantics alone are expressive enough to implement any order over all transactions interacting with the contract. The benefit of having a finite, pre-specified list of call sequences π_C is that we expect application designers to have a good idea of the common usage patterns of their contracts, without wanting to specify semantics over transaction ordering outside of that set (other than the priority of the common patterns).

Call sequence determination. $\sigma(T)$ is determined by execution: it is the projection of T 's execution trace onto calls to contract C , in execution order. For a given transaction and pre-state, $\sigma(T)$ is deterministic, so all correct validators agree. The block builder may need to simulate execution to determine $\sigma(T)$ before ordering; for the lazy evaluation path (Section 8.4), this is moot since the contract's batch-processing function handles ordering internally. For direct execution, transaction metadata (pre-committed call sequences, analogous to access lists) mitigates: transactions that diverge from their declared sequence revert.

Policy examples.

- **Cancel prioritization (Hyperliquid):** $\pi_C = [(\text{Cancel}), (\text{PostOnly}), (\text{GTC}), (\text{IOC})]$. Single-function call sequences. All pure cancel transactions precede all post-only transactions, which precede all GTC transactions, etc. A transaction calling $[\text{Cancel}, \text{GTC}]$ (a multi-call sequence) is unrecognized and unconstrained unless the contract explicitly adds that pattern to π_C . Deployed in production [7].
- **Oracle freshness:** $\pi_C = [(\text{Update}), (\text{Read})]$. Updates precede reads. A transaction that does $[\text{Update}, \text{Read}]$ in a single call could be recognized at a specific priority or left unconstrained, at the contract's discretion.
- **Lending with state-dependent tiebreaker:** A lending contract recognizes:

$$\pi_C = [(\text{Liquidate}), (\text{Liquidate}, \text{Repay}), (\text{Borrow})]$$

Pure liquidations first, then liquidate-and-repay combos, then pure borrows. The contract declares a tiebreaker τ_1 on the (Liquidate) priority level: $\tau_1(S, T_a, T_b)$ orders liquidation transactions by the collateral ratio of their target position in finalized state S , lowest ratio first. The call-sequence priority is static; the intra-priority ordering is state-dependent.

- **Opt-out:** A contract with $\pi_C = []$ (empty list) imposes no ordering constraints. All transactions are unrecognized and the proposer has full discretion.

8.3 MEV Surface Analysis

Ordering guarantees constrain sequence-based MEV for contracts that adopt it and are accessed via direct (non-wrapped) transactions. The residual MEV surface includes:

Intra-application ordering MEV: constrained. Sandwich attacks, frontrunning, and back-running require the attacker to control the relative ordering of their transactions with respect to the victim’s within the same contract. When the ordering is determined by π_C , the proposer cannot insert attacker transactions at strategic positions in the per-contract subsequence. However, the proposer retains discretion over inter-application ordering (where the C -targeted subsequence sits within the block), inclusion (which transactions targeting C enter the block), and interleaving (how transactions targeting different contracts are interspersed).

Exclusion-based MEV: residual. The proposer can selectively exclude transactions (subject to inclusion constraints). Ordering constraints apply to the included subset; the proposer chooses what subset to include. A proposer who dislikes the ordering constraint specified by π_C can selectively exclude specific transactions that aren’t subject to the inclusion constraints to attempt to obtain a more favorable sequence from the remaining included transactions. For example, consider a borrow/lend protocol that has an ordering constraint requiring that oracle updates precede liquidations. If there is no inclusion constraint on the oracle update, a proposer can side-step the ordering constraint by simply only including the liquidation transactions.

Cross-contract MEV: residual and open. When an MEV opportunity spans multiple contracts with different ordering policies, the attacker exploits the interaction between policies rather than the ordering within a single policy. Cross-contract MEV is not addressed by ordering guarantees and is an open problem (Section 8.4).

Ordering guarantees constrain sequencing-dependent extraction on directly-accessed adopting contracts. MEV does not disappear; it migrates from the ordering surface to the inclusion and composability surfaces [28].

8.4 Block Building Challenges and Mitigations

Single- vs. multi-contract ordering. For transactions that each target a single contract with a registered ordering policy, the ordering is determined by π_C . Block validation is straightforward: the validator checks during execution that the per-contract subsequence matches π_C , rejecting the block on any violation. When transactions interact with multiple contracts, each with its own ordering policy, the block building problem becomes combinatorially harder. Bahrani and Neuder [7] provide worked examples with diagrams illustrating each of the following challenges.

Mutual exclusion from heterogeneous policies. Consider two contracts, each with a cancel-prioritization policy (cancels before takes). A transaction T_1 performs **Cancel**(A) + **Swap**(B), and a transaction T_2 performs **Cancel**(B) + **Swap**(A). Placing T_1 before T_2 satisfies A ’s ordering (**Cancel** before **Swap**) but violates B ’s (**Swap** before **Cancel**). The reverse violates A ’s policy. Neither ordering is valid; the transactions are mutually exclusive despite arising from natural user behavior (Alice and Bob each removing stale LP positions and arbitraging the other pool) [7].

The same pattern arises across contract types. An Oracle requiring **Update** before **Read** and a DEX requiring **Cancel** before **Swap**: a transaction that does **Update** + **Swap** and a transaction that does **Read** + **Cancel** cannot coexist in the same block. With two transactions, the resolution is simple: exclude one. With N transactions touching M contracts with heterogeneous policies, determining which subset can coexist and in what order is a combinatorial search over the inclusion and ordering space.

State-dependent transaction behavior. The block building problem is harder when transaction behavior depends on state. If a transaction calls **Remove** or **Swap** depending on the current

pool price, the block builder cannot determine which function the transaction will call without executing it. Static analysis of ordering constraints is insufficient; the builder must simulate execution at each candidate position, and the result may change depending on which other transactions have already executed.

Building vs. validation asymmetry. Verifying ordering conjuncts is cheap: the validator executes the block and checks function-call ordering dynamically. Block building is hard: the proposer must find a valid ordering across all constraints before execution, which is NP-hard in the general case (by reduction from constrained job scheduling with precedence constraints [41]). This asymmetry means ordering guarantees do not meaningfully increase validation cost but do increase block building complexity.

Bypass via wrapping. The outermost-call-target heuristic is trivially bypassable: an attacker wraps their transaction in an arbitrary attacker-controlled outermost contract (with no ordering policy), which then calls the victim contract internally. The ordering constraint of the victim contract does not apply because the outermost target is the attacker’s wrapper. This limits the enforceability of ordering guarantees to direct, non-composable interactions. Applications requiring ordering guarantees must restrict their external entry points (e.g., reverting calls from contracts without registered ordering policies).

Transaction metadata as mitigation. Block building complexity can be reduced if transactions pre-commit to the functions they will call (analogous to Ethereum’s access lists for storage slots). Transactions that call undeclared functions revert but still pay gas. This gives the block builder enough information to order transactions without executing them, at the cost of restricting transaction expressiveness: state-dependent routing (e.g., an on-chain aggregator probing multiple DEXes) cannot pre-commit to its function calls. The tradeoff between expressiveness and expected time-to-inclusion is an open design question [7].

General case. A general solution to multi-contract ordering composability would require either (a) a global constraint solver in the block construction path, (b) cooperative registration between contracts, or (c) a weaker consistency model where per-contract ordering is best-effort for cross-contract calls.

8.4.1 Finalized-State and Non-Finalized-State Ordering

The challenges above concern multi-contract composability: whether transactions touching multiple contracts can coexist under heterogeneous policies. An orthogonal question is whether ordering policies can depend on chain state. Two cases are relevant:

1. *Finalized-state ordering.* π_C and its tiebreakers τ_i depend on finalized (committed) state. Under instant deterministic finality, this is the pre-state of the current block — known before block construction begins and agreed upon by all validators. The ordering is computable deterministically; block building is tractable. This subsumes the fully static case (where π_C and τ_i do not read state at all) as a degenerate instance.
2. *Non-finalized-state ordering.* The ordering depends on state produced during or after the current block’s execution. This introduces a circular dependency: the ordering determines which transactions execute in what sequence, which determines the state, which determines the ordering. Resolving this circularity in the general case requires either fixed-point computation (expensive) or the lazy evaluation pattern described below, which defers execution to the next block and converts non-finalized state into finalized state at

the cost of one block of latency. Despite the complexity, this feature could be a meaningful improvement for applications. For example, a borrow/lend protocol could ensure that positions can only be liquidated if the conditions for the liquidation are sustained beyond the instantaneous state (e.g., to protect against flash-loan style malicious liquidations).

Finalized-state ordering: the Monotone Priority System. For cross-contract composability under finalized-state ordering, Durvasula [8] shows that the *Monotone Priority System*, in which each function call is assigned a global integer priority subject to the constraint that a call’s priority cannot exceed that of any call it references, is the unique system satisfying five natural axioms (existence of valid blocks, baseline per-contract priority, propagation through references, reducibility, and independence of irrelevant calls). Block building reduces to sorting by priority ($O(n \log n)$), resolving the NP-hardness of the general case. The Extension axiom addresses the wrapping bypass identified above: ordering constraints propagate through the call graph, preventing circumvention via wrapper contracts. Because the MPS operates on finalized state, it accommodates both static priorities and state-dependent tiebreakers — any ordering computable from the committed pre-state is compatible.

Non-finalized-state ordering: lazy evaluation via inclusion guarantees. When the ordering depends on state produced during the current block’s execution, the lazy evaluation pattern breaks the circularity. User transactions targeting contract C do not execute application logic directly; they append to a buffer stored in C ’s state. An inclusion trigger (Section 7.1) fires at the end of each block with predicate $P(S) = (\text{buffer}(C) \neq \emptyset)$, invoking C ’s batch-processing function as a single scheduled transaction. This scheduled transaction reads the entire buffer and the current chain state, then executes the buffered operations in any application-defined order. This enables the use cases foreshadowed in Section 8.1 that require reading state at execution time.

This eliminates the multi-transaction ordering problem for the block builder: user transactions are trivial append operations with no meaningful ordering discretion, and the scheduled transaction is a single transaction whose internal execution order is determined by the contract, not by the proposer. The wrapping bypass does not apply (the buffer is internal contract state). NP-hardness does not arise (there is nothing to schedule across contracts; each contract processes its own buffer independently).

The tradeoff is one block of latency. User transactions are not immediately executed; they are buffered and processed in the next cycle. This is the same pattern as asynchronous message queues [7], but implemented natively through Symphony’s inclusion mechanism rather than requiring a separate batching layer. Cross-contract atomicity is also weakened: a user who wants to atomically interact with two buffered contracts cannot do so within a single block, since each contract’s buffer is processed by its own scheduled transaction.

Design space summary. The design space for ordering guarantees has two tractable points and one open problem: (1) finalized-state ordering via the Monotone Priority System [8] with optional state-dependent tiebreakers (zero latency, any ordering computable from committed state), (2) non-finalized-state ordering via buffered lazy evaluation with inclusion guarantees (one-block latency, arbitrary ordering including current-block state), and (3) direct non-finalized-state ordering in the block construction path (zero latency, circular dependency, open). Points (1) and (2) cover the majority of practical use cases.

9 Safety and Liveness

This section establishes Symphony’s safety and liveness properties under standard BFT assumptions. We analyze each execution mode and verification mechanism separately, then compose

them.

9.1 Safety

Safety means that no correct replica accepts an invalid state transition. We decompose safety by execution mode.

9.1.1 Replicated Execution

Lemma 9.1 (Replicated Execution Safety). *For deterministic state transition functions executed in the replicated path, Symphony inherits the safety of the underlying BFT SMR protocol: if fewer than $f < n/3$ replicas are Byzantine, no two correct replicas commit different state transitions for the same block height.*

Proof. Replicated execution follows standard SMR [73]. Each correct replica independently evaluates the deterministic state transition function. By the quorum intersection property ($2 \cdot 2n/3 > n$), any two quorums share at least one correct replica. A correct replica votes for a unique state transition at each height. The proof is identical to Castro and Liskov [25] (for PBFT) or the relevant proof for the underlying SMR protocol. $\square$

9.1.2 Delegated Execution: Single Proof System

Lemma 9.2 (Single-System Delegated Safety under Full-Network Verification). *For a delegated computation verified by a single non-interactive proof system V_i with soundness parameter s_i , where every correct validator independently runs $\text{Verify}(x, \pi_i)$, the probability that an incorrect result is accepted is at most s_i .*

Proof. An incorrect result y^* is accepted only if Verify accepts an invalid proof. By the soundness definition (Definition 2.3), this occurs with probability $\leq s_i$. Since every correct validator independently verifies, no trust delegation is involved: the safety guarantee is purely the proof system’s soundness parameter. $\square$

For the proof systems relevant to Symphony, full-network verification is practical. TEE attestation verification is a signature check (milliseconds). SNARK verification is $O(1)$ in proof size. Neither requires specialized hardware. Every validator can and should verify every top-level proof.

Committee-based shard verification. When proof generation uses the shard decomposition of Section 4.2.2, a single computation may produce k shard proofs plus boundary-linking data. Full-network verification of every shard is redundant: each shard proof is valid or invalid regardless of how many nodes check it. For shard-level verification, committees reduce per-validator work at the cost of an additional failure mode:

$$\Pr[\text{shard incorrectly accepted}] \leq s_i + \Pr[\text{committee compromised}]$$

where the committee compromise probability is bounded by the hypergeometric analysis in Section 9.1.5. Validators not on a shard’s committee accept the committee’s attestation without independently verifying that shard. This is a trust-model change from full-network verification and applies only at the shard level, not at the top-level proof-system level.

9.1.3 Delegated Execution: Multiple Independent Proof Systems

Under full-network verification of each proof system, the composed safety analysis is clean:

Lemma 9.3 (Composed Safety for Independent Systems). *Let $V_1, \dots, V_m$ be proof systems with independent failure modes (Assumption 9.4 below) and soundness parameters $s_1, \dots, s_m$. Under full-network verification, for an application requiring verification by all m systems, the probability that an incorrect result is accepted is at most:*

$$\Pr[\text{incorrect accepted}] \leq \prod_{i=1}^m s_i$$

Proof. An incorrect result is accepted at lattice position $\top = (1, \dots, 1)$ only if it passes verification under every proof system. Under full-network verification, each system's acceptance depends only on its own soundness. For independent systems, the product bound follows. $\square$

Assumption 9.4 (Independence). *Soundness failure of V_i (the proof system accepts an invalid statement) is independent across i . This holds when the proof systems have different soundness bases (e.g., hardware integrity vs. cryptographic assumption) and do not share implementation infrastructure. In practice, shared host infrastructure (hypervisors, compilers, OS kernels) can correlate failures that are nominally orthogonal. The product bound is an idealized limit; the correlation structure must be assessed per deployment.*

With committee-based shard verification. If shard-level committees are used (Section 9.1.5), the per-system acceptance probability becomes $s_i + p_i$ (soundness failure or committee compromise), and the composed bound becomes $\prod_{i=1}^m (s_i + p_i)$. In this regime, committee compromise probabilities p_i may dominate the soundness terms, and committee sizing becomes the binding constraint for practical safety.

9.1.4 Delegated Execution: Correlated Proof Systems

When proof systems have correlated failure modes (e.g., multiple hardware attestations from the same vendor), the product bound does not apply.

Lemma 9.5 (Correlated Safety Bound). *Let $V_1, \dots, V_m$ be proof systems with joint soundness failure probability $s_{1,\dots,m}$ (the probability that all systems simultaneously accept an invalid proof). For an application requiring all m systems:*

$$\Pr[\text{incorrect accepted}] \leq s_{1,\dots,m} + \Pr[\text{all committees compromised}]$$

where $s_{1,\dots,m} \leq \min_i s_i$ but may be much larger than $\prod_i s_i$ if failures are positively correlated.

The joint analysis requires explicit modeling of the correlation structure, standard in reliability theory [10]. For the common case of one hardware attestation vendor and one cryptographic proof system, the two systems have independent failure modes: hardware compromise and cryptographic assumption failure are unrelated events, so the product bound (Theorem 9.3) applies.

9.1.5 Distributed Verification: Hypergeometric Committee Bounds

We derive the probability that a randomly sampled committee contains a Byzantine majority.

Setup. The validator set has n members, of which at most f are Byzantine. A committee of size k is sampled uniformly at random (without replacement) from the validator set. The committee uses a simple majority quorum: a decision is made when $\lceil k/2 \rceil + 1$ members agree.

The number of Byzantine members in a committee of size k follows a hypergeometric distribution:

$$X \sim \text{Hypergeometric}(N = n, K = f, n = k)$$

where N is the population size, K is the number of “defective” items (Byzantine validators), and n is the sample size.

Definition 9.6 (Committee Compromise). *A committee is compromised if it contains a Byzantine majority: $X > k/2$.*

Lemma 9.7 (Committee Safety Bound). *The probability that a committee of size k sampled from n validators with $f < n/3$ Byzantine members contains a Byzantine majority is:*

$$\Pr[X > k/2] = \sum_{x=\lceil k/2 \rceil + 1}^{\min(k, f)} \frac{\binom{f}{x} \binom{n-f}{k-x}}{\binom{n}{k}}$$

Proof. Direct application of the hypergeometric probability mass function. The sum ranges over the possible numbers of Byzantine committee members that constitute a majority. $\square$

Observations.

1. Committee safety improves with committee size k , at a rate that depends on the adversary fraction f/n . The qualitative trend (larger k yields lower compromise probability) is robust; the quantitative values depend on the recomputed table.
2. The hypergeometric distribution (sampling without replacement) gives tighter bounds than the binomial approximation (sampling with replacement), especially when k is a non-negligible fraction of n .
3. These bounds assume uniform staking ($w_i = W/n$ for all i). Non-uniform staking requires weighted sampling analysis, e.g., via Wallenius’ noncentral hypergeometric distribution. The safety guarantee does not “degrade gracefully” in general: heavy-tailed stake distributions can create non-linear vulnerability to committee capture, and the uniform-staking bounds should not be extrapolated to non-uniform settings without explicit analysis.
4. These bounds assume a non-adaptive adversary: committee membership is determined before the adversary decides whom to corrupt. If committee assignments are public and the adversary is adaptive (can target specific committee members after learning the assignment), the hypergeometric bounds do not apply. Cryptographic sortition (as in Algorand [43]) mitigates adaptive corruption; Symphony’s deterministic assignment does not provide this property (see Section 4.3.1).

Comparison to binomial approximation. The binomial approximation treats each committee member as an independent Bernoulli trial with Byzantine probability f/n . This overestimates the committee compromise probability (sampling without replacement reduces variance). For $k \ll n$, the difference is small. For $k/n > 0.1$, the hypergeometric distribution provides meaningfully tighter bounds.

Per-epoch stability. Committee assignments are fixed for an epoch (a configurable number of blocks). Within an epoch, the same committee verifies all proofs for its assigned shard. This amortizes the cost of committee selection and enables caching of verification keys. Between epochs, committees are re-sampled.

Adaptive corruption. The hypergeometric bounds assume a static adversary: the set of Byzantine validators is fixed before committee assignment. An adaptive adversary who can corrupt validators after learning their committee assignment circumvents these bounds entirely. Symphony’s deterministic committee assignment (seeded by epoch randomness) reveals committee membership publicly, creating a window for adaptive targeting. Mitigations: (a) short epochs reduce the corruption window, (b) late-reveal assignment (validators learn their assignment only when they need to verify) reduces the targeting window, (c) cryptographic sortition à la Algorand [43] (validators self-select and prove membership via VRF) eliminates the adaptive-corruption surface but adds protocol complexity. The current design does not implement (c); the hypergeometric bounds should be interpreted as applying under the non-adaptive adversary model only.

9.2 Liveness

Liveness means that every correct transaction eventually makes progress. We analyze liveness along each dimension.

9.2.1 SMR Liveness

Symphony inherits SMR liveness from the underlying BFT protocol [34]. After GST, every correct proposer eventually produces a block that is committed by all correct replicas. The extended validity conjuncts add computational cost to the critical path but do not fundamentally alter the SMR liveness proof, provided the predicate evaluation budget $\alpha \cdot \Delta_B$ is respected (Section 6.4.3).

9.2.2 Executor Liveness

A delegated computation may stall if the executor crashes or refuses to produce a result.

Definition 9.8 (Executor TTL). *Each async commit includes a time-to-live TTL: the number of blocks within which the executor must submit a settlement transaction.*

If the TTL expires without settlement:

1. The computation status transitions from Pending to Expired.
2. The executor’s bond for that computation is partially slashed (to compensate for wasted block space and user delay).
3. The computation is eligible for rescheduling: a new executor can claim it via a standard transaction.

Liveness guarantee: every computation that is committed will eventually either settle (correct result with proofs) or expire and be rescheduled, provided at least one willing and capable executor exists in the network.

TTL and partial synchrony. The TTL mechanism creates a tension with the partial synchrony model. Before GST, messages can be delayed arbitrarily. An honest executor who computes a correct result and submits a valid settlement transaction may have the transaction delayed past the TTL by a network partition—resulting in slashing of an honest executor. This is a synchrony assumption hidden in the TTL mechanism: the executor must be able to deliver the settlement transaction within TTL blocks of committed chain time.

Mitigation. The TTL should be set conservatively (much larger than the expected computation + proof generation time) to account for network delays. Additionally, the protocol can tie TTL expiry to committed state rather than wall time: the TTL counts committed blocks, not elapsed real time, so the executor has TTL committed blocks to submit. Since committed blocks require SMR (which resumes after GST), the TTL clock effectively pauses during partitions that prevent block production. However, if blocks continue to be produced by a partition that excludes the executor, the executor can still be slashed despite acting honestly. This is an inherent limitation of timeout-based mechanisms in partially synchronous systems.

9.2.3 Prover Liveness

After the executor submits a result, proof generation for each proof system proceeds independently. A proof system may fail to produce a proof within the expected latency (prover crash, resource exhaustion, proof system bug).

The result remains at its current lattice position. Applications that require a higher lattice position than currently achieved simply wait. If the proving deadline expires:

1. The proof escrow for that proof system is returned to the payer (no penalty for the prover, as proof failure is not necessarily Byzantine behavior).
2. The result’s lattice position is frozen at its current state.
3. Applications requiring the unachieved proof system dimension must either (a) wait for a re-proving attempt, (b) fall back to a lower safety requirement, or (c) trigger a compensating action via the saga mechanism.

9.2.4 Liveness Under Inclusion & Exclusion Guarantees

Forced inclusion and exclusion predicates are evaluated in the consensus critical path. Under the protocol cap $k_{\max}$, the total evaluation cost is bounded: $\sum_{i=1}^k c_i \leq k_{\max} \cdot c_{\max} \leq \alpha \cdot \Delta_B$, where $c_{\max}$ is the maximum per-predicate evaluation cost (enforced by the deposit/slash mechanism). Since predicates are independent, evaluation parallelizes across P cores (Section 6.4.3).

Block construction under inclusion and exclusion enforcement is efficient. Given the set of active triggers and their evaluation results, the proposer constructs a valid block as follows: (a) include all transactions mandated by triggered inclusion predicates, (b) exclude all transactions matched by active exclusion triggers, (c) if a transaction is mandated by an inclusion trigger and matched by an exclusion trigger, apply the conflict resolution policy (Section 7.3)—by default, inclusion takes precedence. Each step is linear in the number of active triggers. The conflict resolution policy is a static priority rule evaluated in $O(1)$ per conflict, so runtime conflict resolution does not add combinatorial complexity. This guarantees the existence of a computationally efficient procedure for producing a valid block under active inclusion/exclusion triggers. We make no claim about the *economic* efficiency of the resulting block—revenue maximization under guarantee constraints is a separate mechanism design problem, orthogonal to liveness, and is the subject of ongoing work.

Soundness vs. completeness of conflict detection. Conflict detection at registration time (Section 7.3) is sound but not complete: it may reject non-conflicting trigger registrations (false positives) but will never allow a registration that creates an unsatisfiable validity predicate (no false negatives). Every set of active triggers admitted by the registration-time check has a valid block—the block construction procedure above always succeeds. The incompleteness is conservative: some safe trigger combinations are rejected, but no admitted combination causes a liveness failure.

9.2.5 Liveness Under Ordering Guarantees

Application-defined ordering adds a constraint to block construction but does not prevent block production: the proposer can always order transactions according to π_C (deterministic function of the transaction set and state). The cost is $O(|T_C| \log |T_C|)$ per contract C with an ordering policy, where $|T_C|$ is the number of transactions targeting C in the block.

10 Discussion

10.1 Limitations

Predictable validity. Symphony’s extended external validity requires predictable validity: the proposer can exactly compute the state resulting from confirmation of their block (Theorem 2.9). This limits compatibility to SMR protocols that provide this property, excluding or limiting protocols with high staleness. DAG-based protocols can use bounded-staleness predicates (Corollary 6.2), but this weakens the precision of inclusion/exclusion triggers.

Non-interactivity excludes fraud proofs. The restriction to non-interactive proof systems (Section 2.3) excludes fraud-proof-based verification, forgoing the efficiency advantage of optimistic schemes (proofs are generated only upon dispute). The benefit — no synchrony assumptions in verification, bounded verification latency, simpler P2P layer — was judged to outweigh the cost for the target workload class. For workloads where fraud proofs would be significantly more efficient (e.g., general EVM execution), this constraint may not be appropriate.

Predicate evaluation overhead. Extended external validity adds computational cost to the block production critical path. The censorship-resistance/liveness tradeoff (Section 6.4.3) is fundamental: stronger censorship resistance comes at the expense of liveness. However, predicate evaluation is embarrassingly parallel (Section 6.4.3): inclusion, exclusion, and ordering predicates are independent, and the effective budget scales as $P \cdot \alpha \cdot \Delta_B$ with P parallel evaluation slots. The protocol cap $k_{\max}$ bounds total work, but the wall-clock cost scales as $O(k_{\max}/P)$, making the overhead practical for modern multi-core validators.

Inclusion/exclusion conflict resolution. The interaction between forced inclusion and forced exclusion introduces a satisfiability problem (Proposition 6.4). Full conflict detection is undecidable in the general case. The practical approach (static analysis + registration-time evaluation + runtime resolution policy) is sound but not complete: it may reject non-conflicting registrations.

Verification window for high-latency proof systems. For large workloads, SNARK proving may take hours. During this verification window, safety depends on the hardware attestation and the economic safety model. This is a constraint of current proof technology. As proof systems improve, the window narrows; the architecture accommodates this progression without structural change.

Multi-contract ordering composability. Symphony’s ordering policies are well-defined for single-contract transactions. The stateless case (function-identity policies) is resolved by the Monotone Priority System [8]. The stateful case is resolved via buffered lazy evaluation with inclusion-triggered batch processing at the cost of one block of latency and reduced cross-contract atomicity (Section 8.4). Direct stateful ordering in the block construction path (zero latency, full composability) remains NP-hard and open.

Stale preconditions in delegated execution. The saga model (Section 3.4) does not enforce read-set consistency between async commit and settlement. State may change between commit (block N) and settlement (block $N + k$), invalidating the assumptions under which the delegated computation was performed. The protocol provides compensating actions but does not prevent settlement based on stale preconditions by default. Applications must implement their own read-set validation in the settlement transaction. This is a deliberate design choice (avoiding locks on shared state) with a known footgun for applications that depend on volatile state during the async gap.

Hypergeometric bounds require large validator sets and uniform staking. The committee safety bounds (Theorem 9.7) assume uniform staking and a non-adaptive adversary. They are meaningful only when $n \gg 100$. For small validator sets, committees must be large relative to n , reducing the efficiency benefit of distributed verification. For non-uniform staking (the norm in production PoS systems), the bounds require extension to weighted sampling (e.g., Wallenius’ noncentral hypergeometric distribution), and the safety guarantees may degrade non-linearly under heavy-tailed stake distributions.

Workload decomposition requires structure. Structure-exploiting decomposition (Section 4.2) relies on the algebraic structure of the computational object. Unstructured computations (e.g., a large arbitrary boolean circuit) do not benefit from this approach and must use generic decomposition, which is less efficient.

Correlated proof system failures. The product lattice provides composed safety for independent proof systems, but correlated failures (e.g., a vulnerability affecting all hardware vendors simultaneously) can compromise multiple lattice dimensions. The degradation protocol (Section 5.8) mitigates this but cannot eliminate systemic risk.

Adaptive adversaries and committee sampling. The hypergeometric committee safety bounds (Section 9.1.5) assume a non-adaptive adversary whose Byzantine validators are fixed before committee assignment. If the adversary is adaptive and can corrupt or deny service to specific validators after learning their committee assignment, the bounds do not apply. Symphony’s deterministic committee assignment reveals membership publicly, creating a window for targeted corruption. Cryptographic sortition (as in Algorand [43]) mitigates this but is not implemented in the current design. The committee safety guarantees should be interpreted as holding under the non-adaptive adversary model; extending them to adaptive adversaries requires protocol changes.

11 Conclusion

Replicated state machine execution fails for the workload class that motivates this paper. Hardware non-reproducibility across GPU architectures makes bit-exact agreement impossible for parallel floating-point computations, regardless of shared randomness. Algorithmic randomness in randomized inference compounds the problem. The transition from SMR-performance optimization to execution-aware SMR is architectural.

Symphony responds with a single organizing principle: disaggregate the block proposer’s powers and reassign them individually via extended external validity. Inclusion, exclusion, and sequencing rights are transferred from the proposer to the protocol, users, and applications through three mechanisms, each enforced as a conjunct of the block validity function in the SMR critical path. The enforcement is deterministic and parallelizable across predicates.

Delegated execution operates in superposition with replicated execution over shared state. The sealed-seed randomness model binds the executor to a user-provided seed encrypted to the

TEE, preventing grinding without making the output predictable. Proof generation exploits the algebraic structure of the computational objects: the boundary width at partition points determines the linking cost, and narrow boundaries between architectural components (transformer blocks, convolutional stages) yield efficient decompositions. The prover-friendly design point — optimizing proving latency rather than proof size, since proofs are verified by committees off-chain rather than enshrined on-chain — inverts the standard ZKP tradeoff and shrinks the verification window between hardware attestation and cryptographic finality.

For application-defined ordering, the stateless case is resolved by the Monotone Priority System [8], which is the unique axiomatically justified contract-specific sequencing system under five natural axioms. The stateful case is resolved by composing inclusion-guaranteed batch execution with a one-block-delay buffer, converting arbitrary state-dependent ordering into a single-transaction execution problem. Direct zero-latency stateful ordering in the block construction path remains NP-hard and open.

The process of building and distributing a block is larger than what is explored here. Beyond inclusion, exclusion, and ordering guarantees, there exist other dimensions such as proposal timing, blob/block data dissemination, the right to fees, and priority guarantees. The formal results — predictable validity, the censorship-resistance/liveness tradeoff, soundness composition, and the hypergeometric committee bounds — collectively establish conditions under which execution-aware SMR is safe, live, and practical.

References

- [1] M. Al-Bassam. LazyLedger: A distributed data availability ledger with client-side smart contracts. arXiv:1905.09274, 2019.
- [2] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, J. Yellick. Hyperledger Fabric: A distributed operating system for permissioned blockchains. In *Proc. 13th EuroSys Conf.*, pp. 1–15, 2018.
- [3] Offchain Labs. Arbitrum. 2021.
- [4] L. Babai. Trading group theory for randomness. In *STOC*, 1985.
- [5] M. Bahrani, P. Garimidi, T. Roughgarden. Transaction fee mechanism design with active block producers. arXiv:2307.01686, 2023.
- [6] M. Bahrani, N. Durvasula. Resonance: Transaction fees for heterogeneous computation. arXiv:2411.11789, 2024.
- [7] Ethereum Research. Application-controlled execution: A case study on cancel prioritization. <https://ethresear.ch/t/application-controlled-execution-a-case-study-on-cancel-prioritization/23977>.
- [8] N. Durvasula. The Monotone Priority System: Foundations of contract-specific sequencing. arXiv:2601.20783, 2026.
- [9] P. Bailis, S. Venkataraman, M. J. Franklin, J. M. Hellerstein, I. Stoica. Probabilistically bounded staleness for practical partial quorums. *Proc. VLDB Endow.* 5(8):776–787, 2012.
- [10] R. E. Barlow, F. Proschan. *Statistical Theory of Reliability and Life Testing*. 1975.

- [11] M. Ben-Or, S. Goldwasser, A. Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Proc. 20th ACM Symp. Theory of Computing (STOC)*, pp. 1–10, 1988.
- [12] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. IACR ePrint 2018/046, 2018.
- [13] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O’Neil, P. O’Neil. A critique of ANSI SQL isolation levels. In *Proc. ACM SIGMOD Int. Conf. Management of Data*, pp. 1–10, 1995.
- [14] P. A. Bernstein, V. Hadzilacos, N. Goodman. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.
- [15] G. Birkhoff. *Lattice Theory*. AMS, 3rd ed., 1967.
- [16] B. H. Bloom. Space/time trade-offs in hash coding. *CACM*, 13(7), 1970.
- [17] M. Blum, P. Feldman, S. Micali. Non-interactive zero-knowledge and its applications. In *Proc. 20th ACM Symp. Theory of Computing (STOC)*, pp. 103–112, 1988.
- [18] J. Bootle, A. Cerulli, P. Chaidos, J. Groth, C. Petit. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In *Proc. EUROCRYPT*, pp. 327–357, 2016.
- [19] S. Bowe, A. Gabizon, M. Mihaescu. Recursive proof composition from accumulation schemes. IACR ePrint 2019/1021, 2019.
- [20] E. A. Brewer. Towards robust distributed systems. *PODC Keynote*, 2000.
- [21] E. Buchman, J. Kwon, Z. Milosevic. The latest gossip on BFT consensus. arXiv:1807.04938, 2018.
- [22] V. Buterin, D. Hernandez, T. Kamphefner, K. Pham, Z. Qiao, D. Ryan, J. Sin, Y. Wang, Y. X. Zhang. Combining GHOST and Casper. arXiv:2003.03052, 2020.
- [23] V. Buterin. Two-slot proposer-builder separation. Ethereum Research, <https://ethresear.ch/t/two-slot-proposer-builder-separation/10980>, 2021.
- [24] C. Cachin, K. Kursawe, V. Shoup. Random oracles in Constantinople: Practical asynchronous Byzantine agreement using cryptography. *J. Cryptology* 18(3):219–246, 2005.
- [25] M. Castro, B. Liskov. Practical Byzantine fault tolerance. In *Proc. 3rd Symp. Operating Systems Design and Implementation (OSDI)*, pp. 173–186, 1999.
- [26] Celestia Labs. Celestia. 2022.
- [27] P. Cousot, R. Cousot. Abstract interpretation. In *POPL*, 1977.
- [28] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, A. Juels. Flash Boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *Proc. IEEE Symp. Security and Privacy*, pp. 910–927, 2020.
- [29] G. Danezis, L. Kokoris-Kogias, A. Sonnino, A. Spiegelman. Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus. In *Proc. 17th EuroSys Conf.*, pp. 34–50, 2022.
- [30] U. Dayal, D. R. McCarthy. The HiPAC project: Combining active databases and timing constraints. *SIGMOD Record* 17(1):51–70, 1988.
- [31] J. Dean, S. Ghemawat. MapReduce. In *OSDI*, 2004.

- [32] J. Dean, G. Corrado, R. Monga, K. Chen, M. Devin, M. Mao, A. Ranzato, A. Senior, P. Tucker, K. Yang, Q. V. Le, A. Y. Ng. Large scale distributed deep networks. In *Proc. NeurIPS*, pp. 1232–1240, 2012.
- [33] D. E. Denning. A lattice model of secure information flow. *CACM*, 19(5), 1976.
- [34] C. Dwork, N. Lynch, L. Stockmeyer. Consensus in the presence of partial synchrony. *J. ACM* 35(2):288–323, 1988.
- [35] Ethereum Foundation. EIP-7547: Inclusion lists. 2023.
- [36] Espresso Systems. Espresso Sequencer. 2023.
- [37] R. Fagin, J. Y. Halpern, Y. Moses, M. Y. Vardi. *Reasoning About Knowledge*. MIT Press, 1995.
- [38] Flashbots. 2021.
- [39] Flashbots JSON-RPC Endpoints. 2026. https://docs.flashbots.net/flashbots-auction/advanced/rpc-endpoint#eth_sendbundle
- [40] H. Garcia-Molina, K. Salem. Sagas. In *Proc. ACM SIGMOD Int. Conf. Management of Data*, pp. 249–259, 1987.
- [41] M. R. Garey, D. S. Johnson. *Computers and Intractability*. 1979.
- [42] R. Gennaro, C. Gentry, B. Parno. Non-interactive verifiable computing: Outsourcing computation to untrusted workers. In *Proc. CRYPTO*, pp. 465–482, 2010.
- [43] Y. Gilad, R. Hemo, S. Micali, G. Vlachos, N. Zeldovich. Algorand: Scaling Byzantine agreements for cryptocurrencies. In *Proc. 26th ACM Symp. Operating Systems Principles (SOSP)*, pp. 51–68, 2017.
- [44] S. Gilbert, N. Lynch. Brewer’s conjecture. *SIGACT News*, 33(2), 2002.
- [45] O. Goldreich, S. Micali, A. Wigderson. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *J. ACM* 38(3):691–729, 1991.
- [46] S. Goldwasser, S. Micali, C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM J. Comput.* 18(1):186–208, 1989.
- [47] S. J. Grossman, O. D. Hart. Costs and benefits of ownership. *JPE*, 94(4), 1986.
- [48] J. Groth. On the size of pairing-based non-interactive arguments. *EUROCRYPT*, 2016.
- [49] M. A. Harrison, W. L. Ruzzo, J. D. Ullman. Protection in operating systems. *Commun. ACM* 19(8):461–471, 1976.
- [50] O. Hart, J. Moore. Property rights and the nature of the firm. *JPE*, 98(6), 1990.
- [51] M. P. Herlihy, J. M. Wing. Linearizability. *ACM TOPLAS*, 12(3), 1990.
- [52] R. Housley, W. Ford, W. Polk, D. Solo. Internet X.509 public key infrastructure certificate and CRL profile. RFC 2459, 1999.
- [53] L. Hurwicz. On informationally decentralized systems. 1972.
- [54] H. Jooybar, W. W. L. Fung, M. O’Connor, J. Devietti, T. M. Aamodt. GPUDET: A deterministic GPU architecture. In *Proc. ASPLOS*, pp. 1–12, 2013.

- [55] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borch, E. Boyle, P. Boyle, J. P. Cantin, C. Chao, C. Clark, J. Corriel, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmamghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khatamian, J. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snelham, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, D. H. Woo. In-datacenter performance analysis of a tensor processing unit. In *Proc. 44th Int. Symp. Computer Architecture (ISCA)*, pp. 1–12, 2017.
- [56] A. Kate, G. M. Zaverucha, I. Goldberg. Constant-size commitments to polynomials and their applications. In *Proc. ASIACRYPT*, pp. 177–194, 2010.
- [57] M. Kelkar, F. Zhang, S. Goldfeder, A. Juels. Order-fairness for Byzantine consensus. In *Proc. CRYPTO*, pp. 451–480, 2020.
- [58] M. Kelkar, S. Deb, S. Long, A. Juels, S. Kannan. Themis: Fast, strong order-fairness in Byzantine consensus. IACR ePrint 2021/1465, 2021.
- [59] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM* 21(7):558–565, 1978.
- [60] Anza. Alpenglow: Solana consensus upgrade. 2025.
- [61] B. Y. Chan, R. Pass. Simplex consensus: A simple and fast consensus protocol. IACR ePrint 2023/463, 2023.
- [62] B. K. Chou, A. Lewis-Pye, P. O’Grady. Minimmit: Fast finality with even faster blocks. arXiv:2508.10862, 2025.
- [63] M. M. Jalalzai, K. Babel, J. Komatovic, T. Klenze, S. Das, F. Elsheimy, M. Setrin, J. Bergschneider, B. Gilkalaye. MonadBFT: Fast, responsive, fork-resistant streamlined consensus. arXiv:2502.20692, 2025.
- [64] R. Motwani, P. Raghavan. *Randomized Algorithms*. Cambridge, 1995.
- [65] S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008.
- [66] Optimism Foundation. 2021.
- [67] A. Ozdemir, D. Boneh. Collaborative zk-SNARKs. *USENIX Security*, 2022.
- [68] A. Ozdaglar et al. Optimistic search on blockchains. arXiv:2506.14768, 2025.
- [69] B. Parno, J. Howell, C. Gentry, M. Raykova. Pinocchio: Nearly practical verifiable computation. In *Proc. IEEE Symp. Security and Privacy*, pp. 238–252, 2013.
- [70] N. W. Paton, O. Diaz. Active database systems. *ACM Comp. Surveys*, 31(1), 1999.
- [71] Team Rocket, M. Yin, K. Sekniqi, R. van Renesse, E. G. Sirer. Scalable and probabilistic leaderless BFT consensus through metastability. arXiv:1906.08936, 2020.
- [72] T. Roughgarden. Transaction fee mechanism design. In *EC*, 2021.

- [73] F. B. Schneider. Implementing fault-tolerant services using the state machine approach: A tutorial. *ACM Comput. Surv.* 22(4):299–319, 1990.
- [74] Shutter Network. 2022.
- [75] A. Spiegelman, N. Giridharan, A. Sonnino, L. Kokoris-Kogias. Bullshark: DAG BFT protocols made practical. In *Proc. ACM SIGSAC Conf. Computer and Communications Security*, pp. 2705–2718, 2022.
- [76] Flashbots. SUAVE. 2023.
- [77] D. B. Terry, A. J. Theimer, K. Petersen, A. J. Demers, M. J. Spreitzer, C. H. Hauser. Session guarantees for weakly consistent replicated data. In *Proc. 3rd Int. Conf. Parallel and Distributed Information Systems (PDIS)*, pp. 140–149, 1994.
- [78] E. Tsang. *Foundations of Constraint Satisfaction*. 1993.
- [79] P. Valiant. Incrementally verifiable computation. In *TCC*, 2008.
- [80] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin. Attention is all you need. In *Proc. NeurIPS*, pp. 5998–6008, 2017.
- [81] M. Walfish, A. J. Blumberg. Verifying computations without reexecuting them. *CACM*, 58(2), 2015.
- [82] H. Wu, W. Zheng, A. Chiesa, R. A. Popa, I. Stoica. DIZK: A distributed zero knowledge proof system. In *Proc. USENIX Security*, pp. 675–692, 2018.
- [83] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, I. Abraham. HotStuff: BFT consensus with linearity and responsiveness. In *Proc. 2019 ACM Symp. Principles of Distributed Computing (PODC)*, pp. 347–356, 2019.
- [84] B. Monnot. Decoupling throughput from local building. *Ethereum Research*, <https://ethresear.ch/t/decoupling-throughput-from-local-building/22004>, 2024.
- [85] B. Monnot. Unbundling staking: Towards rainbow staking. *Ethereum Research*, <https://ethresear.ch/t/unbundling-staking-towards-rainbow-staking/18683>, 2024.
- [86] B. Monnot. Unbundling PBS: Towards protocol-enforced proposer commitments (PEPC). *Ethereum Research*, <https://ethresear.ch/t/unbundling-pbs-towards-protocol-enforced-proposer-commitments-pepc/13879>, 2022.
- [87] M. Bahrani. Beyond Multidimensional Fee Markets. *Talk, Devcon* Available at: <https://www.youtube.com/watch?v=qf51v48KhH0>. 2024.